LeetCode Practice

25 Medium: Matrix

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

25
MEDIUM

Patterns (in order)

1. Matrix (25)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

25 questions | tap a title to jump to it

Medium questions (25)	Go >
Matrix (25)	**Go >**
M 1. Valid Sudoku	LC 36
M 2. Rotate Image	LC 48
M 3. Spiral Matrix	LC 54
M 4. Spiral Matrix II	LC 59
M 5. Unique Paths II	LC 63
M 6. Minimum Path Sum	LC 64
M 7. Set Matrix Zeroes	LC 73
M 8. Search a 2D Matrix	LC 74
M 9. Maximal Square	LC 221
M 10. Search a 2D Matrix II	LC 240
M 11. Game of Life	LC 289
M 12. Range Sum Query 2D - Immutable	LC 304
M 13. Range Sum Query 2D - Mutable	LC 308
M 14. Sparse Matrix Multiplication	LC 311
M 15. Diagonal Traverse	LC 498
M 16. Lonely Pixel I	LC 531
M 17. Candy Crush	LC 723
M 18. Image Overlap	LC 835
M 19. Minimum Falling Path Sum	LC 931
M 20. Find Smallest Common Element in All Rows	LC 1198
M 21. Count Square Submatrices with All Ones	LC 1277
M 22. Matrix Block Sum	LC 1314
M 23. Maximum Number of Points with Cost	LC 1937
M 24. Maximum Matrix Sum	LC 1975
M 25. Increment Submatrices by One	LC 2536

Medium questions (25)

MEDIUM | TOPIC 1 OF 1

Matrix

25 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Valid Sudoku LC 36
2. Rotate Image LC 48
3. Spiral Matrix LC 54
4. Spiral Matrix II LC 59
5. Unique Paths II LC 63
6. Minimum Path Sum LC 64
7. Set Matrix Zeroes LC 73
8. Search a 2D Matrix LC 74
9. Maximal Square LC 221
10. Search a 2D Matrix II LC 240
11. Game of Life LC 289
12. Range Sum Query 2D - Immutable LC 304
13. Range Sum Query 2D - Mutable LC 308
14. Sparse Matrix Multiplication LC 311
15. Diagonal Traverse LC 498
16. Lonely Pixel I LC 531
17. Candy Crush LC 723
18. Image Overlap LC 835
19. Minimum Falling Path Sum LC 931
20. Find Smallest Common Element in All Rows LC 1198
21. Count Square Submatrices with All Ones LC 1277
22. Matrix Block Sum LC 1314
23. Maximum Number of Points with Cost LC 1937
24. Maximum Matrix Sum LC 1975
25. Increment Submatrices by One LC 2536

MEDIUM | Matrix | LeetCode #36 | 1 of 25 | 1/25

Valid Sudoku

QUESTION

Determine if a 9 x 9 Sudoku board is valid. Only the filled cells need to be validated according to the following rules:

CONSTRAINTS

- 1. Each row must contain the digits 1-9 without repetition.
- 2. Each column must contain the digits 1-9 without repetition.
- 3. Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1-9 without repetition.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.

Example 1

Input	`board = [["5","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".","."],[".","9","8",".",".",".",".","6","."],["8",".",".",".","6",".",".",".","3"],["4",".",".","8",".","3",".",".","1"],["7",".",".",".","2",".",".",".","6"],[".","6",".",".",".",".","2","8","."],[".",".",".","4","1","9",".",".","5"],[".",".",".",".","8",".",".","7","9"]]`
Output	`true`

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

QUESTION

Example 2

Input	`board = [["8","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".","."],[".","9","8",".",".",".",".","6","."],["8",".",".",".","6",".",".",".","3"],["4",".",".","8",".","3",".",".","1"],["7",".",".",".","2",".",".",".","6"],[".","6",".",".",".",".","2","8","."],[".",".",".","4","1","9",".",".","5"],[".",".",".",".","8",".",".","7","9"]]`
Output	`false`
Explanation	*Same as Example 1, except with the 5 in the top left corner being modified to 8. Since there are two 8's in the top left 3x3 sub-box, it is invalid.*

- board.length == 9
- board[i].length == 9
- board[i][j] is a digit 1-9 or '.'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Check if a partially filled 9x9 sudoku board is valid — no repeats in any row,

column, or 3x3 box. Ignore empty cells ('.'). Right?

A valid board doesn't have to be solvable — just no current conflicts."

"The two examples differ only in one cell: '8' appears twice in the top-left 3x3 box -> false ok"

Are cell values always within a known range?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    compare it with the other cells in its row,
        its column and its 3 x 3 box
    fail on any repeat
```

TIME	SPACE
O(n^3)	O(1)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Each of the n^2 cells is compared with about 3n others (n = 9, so it is a constant, but the work is repeated). One pass with a set per row, column and box is O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Single pass with three boolean arrays — O(81) = O(1). Walk-through:"

rows[r][d], cols[c][d], boxes[b][d]: True if digit d seen in row r / col c / box b.

Box index: b = (r//3)*3 + (c//3) — maps (r,c) to one of 9 boxes.

For each cell (r,c):

If cell is '.': skip.

`d = int(cell) - 1 (0-indexed digit).`

If rows[r][d] or cols[c][d] or boxes[b][d]: return False (duplicate).

Mark all three as True.

Return True.

WHY three separate arrays: each constraint (row/col/box) tracks digit usage independently.

WHY box formula (r//3)*3 + c//3: divides grid into 3×3 groups starting at corners (0,3,6).

Time: O(1) fixed 9×9. Space: O(1).

PHASE 4 CLEAN CODE

```
class _36_ValidSudoku:
    def isValidSudoku(self, board: List[List[str]]) -> boo
l:
        rows  = [[False] * 9 for _ in range(9)]
        cols  = [[False] * 9 for _ in range(9)]
        boxes = [[False] * 9 for _ in range(9)]   # indexe
d by box_id = (r//3)*3 + c//3

        for r in range(9):
            for c in range(9):
                if board[r][c] == '.':
                    continue
                d      = int(board[r][c]) - 1
                box_id = (r // 3) * 3 + (c // 3)
                if rows[r][d] or cols[c][d] or boxes[box_i
d][d]:
                    return False
                rows[r][d] = cols[c][d] = boxes[box_id][d]
 = True
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace isValidSudoku on a partially filled 9×9 board."

Three checks: rows (no digit repeat), cols (no digit repeat), 3×3 boxes (no repeat).

Use sets. For each cell (r,c) with digit d:

Row check: seen_rows[r].add(d). If already there -> invalid.

Col check: seen_cols[c].add(d).

Box check: box_id=(r//3)*3+(c//3). seen_boxes[box_id].add(d).

PHASE 5 TEST & DEBUG

Single pass O(81). If any add finds duplicate -> return False. Else True ok

Key edge: same digit in row is invalid even if same column has it at different row.

Validation (not placement backtracking here): used={} per row/col/box. path=[cell values]. call(next_cell,path): try digit->if conflict skip (prune). backtrack: undo digit. sudoku solver would use this approach.

Trace: board[0][0]="5" -> r=0,c=0,d=5; seen_rows[0].add(5)ok, seen_cols[0].add(5)ok, seen_boxes[0].add(5)ok — no duplicate, continue.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(81)	**O(81)**
=O(1)	=O(1) — fixed 9x9 board

The box_id formula (r//3)*3+(c//3) maps each of the 9 boxes to 0-8. Know this cold.

Alternative: use a set of tuples like (r,'row',d), (c,'col',d), (box_id,'box',d) — one set.

Given more time I'd ask: implement a Sudoku solver?

That's backtracking — for each empty cell, try 1-9, recurse, backtrack on failure.

MEDIUM | Matrix | LeetCode #48 | 2 of 25 | 2/25

Rotate Image

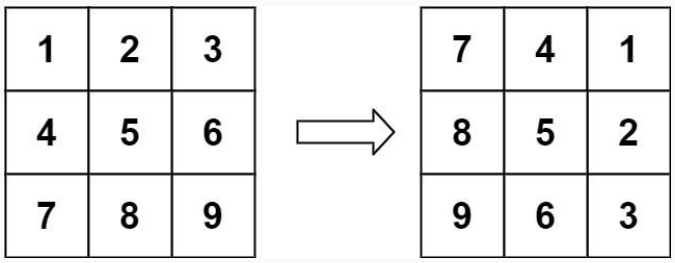
QUESTION

You are given an n x n 2D matrix representing an image, rotate the image by 90 degrees (clockwise).

You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.

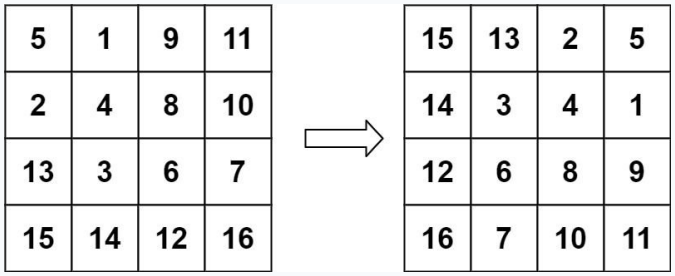
Example 1

Input	`matrix = [[1,2,3],[4,5,6],[7,8,9]]`
Output	`[[7,4,1],[8,5,2],[9,6,3]]`



Example 2

Input	`matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]`
Output	`[[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]`



CONSTRAINTS

- n == matrix.length == matrix[i].length
- 1 <= n <= 20
- 1000 <= matrix[i][j] <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Rotate an n x n matrix 90 degrees clockwise in-place. No extra matrix allowed. Right?

After rotation: element at (r,c) moves to (c, n-1-r)."

"[[1,2,3],[4,5,6],[7,8,9]] -> [[7,4,1],[8,5,2],[9,6,3]] ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
new = an n x n matrix
for j) in j)s:
    new[j][n - 1 - i] = old[i][j]
    copy new back
```

TIME	SPACE
O(n^2)	O(n^2)

WHY IT IS SLOW

Optimal in time; the cost is the second matrix. Transposing in place and then reversing each row rotates it with O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (two-step in-place)

Approach: optimal approach

"Two operations, each reversible and composable:

Step 1: Transpose (swap matrix[r][c] with matrix[c][r]).

Step 2: Reverse each row.

Together these produce a 90° clockwise rotation. O(n²) time, O(1) space.

Verify: (r,c) ->transpose-> (c,r) ->reverse row-> (c, n-1-r). Correct!"

Key insight: transpose + reverse-each-row is algebraically equivalent to a 90° clockwise rotation — two O(n²) in-place passes with O(1) extra space.

PHASE 4 CLEAN CODE

Step 1: transpose

Step 2: reverse each row

```
class _48_RotateImage:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for r in range(n):
            for c in range(r + 1, n):
                matrix[r][c], matrix[c][r] = matrix[c][r],
 matrix[r][c]
        for row in matrix:
            row.reverse()
```

PHASE 5 TEST & DEBUG

"Let me trace rotate([[1,2,3],[4,5,6],[7,8,9]])."

Step 1: transpose (swap i,j and j,i).

[1,2,3] [1,4,7]

[4,5,6] -> [2,5,8]

[7,8,9] [3,6,9]

Step 2: reverse each row.

PHASE 5 TEST & DEBUG

[1,4,7] -> [7,4,1]. [2,5,8] -> [8,5,2]. [3,6,9] -> [9,6,3].

Result: [[7,4,1],[8,5,2],[9,6,3]] ok (90° clockwise rotation).

WHY transpose then reverse: transpose mirrors along main diagonal; reverse row gives clockwise rotation.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	**O(1)** in-place

Counter-clockwise 90°: reverse each row first, then transpose.

180°: transpose then transpose (just reverse all rows, then reverse their order).

The two-step approach is elegant — each step is simple and reversible.

Given more time I'd ask: rotate a non-square matrix?

No longer possible in-place with the same dimensions — need a new m×n matrix.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Matrix | LeetCode #54 | 3 of 25 | 3/25

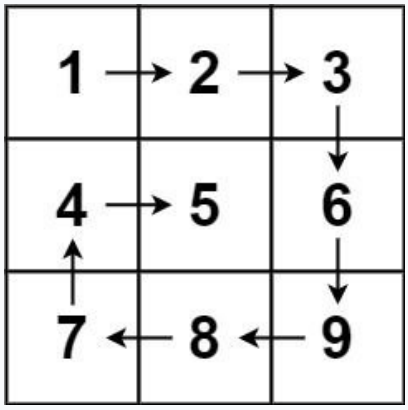
Spiral Matrix

QUESTION

Given an m x n matrix, return all elements of the matrix in spiral order.

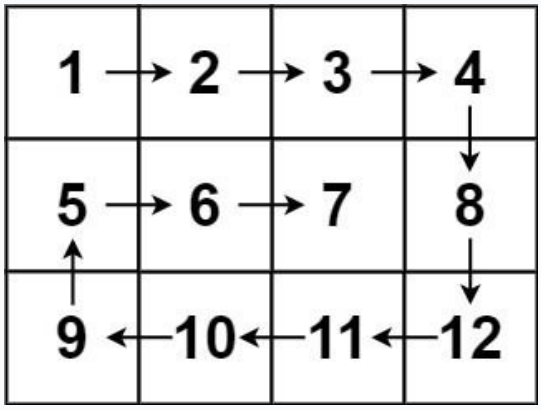
Example 1

Input	`matrix = [[1,2,3],[4,5,6],[7,8,9]]`
Output	`[1,2,3,6,9,8,7,4,5]`



Example 2

Input	`matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]`
Output	`[1,2,3,4,8,12,11,10,9,5,6,7]`



CONSTRAINTS

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 10
- 100 <= matrix[i][j] <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Return all matrix elements in spiral order: right along top, down along right side,

left along bottom, up along left side — then repeat for the inner rectangle. Right?

Works for non-square matrices."

"[[1,2,3],[4,5,6],[7,8,9]] -> [1,2,3,6,9,8,7,4,5] ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
make a visited matrix
start top-left heading right
take a step, turn clockwise when the next cell is out
    of bounds or visited
stop after m * n cells
```

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

Optimal in time; the cost is the visited matrix. Shrinking four boundaries (top, bottom, left, right) peels the matrix in O(m * n) time and O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (layer boundaries)

"Boundary shrink — O(mn). Walk-through:"

top=0, bottom=m-1, left=0, right=n-1. result=[].

While top<=bottom and left<=right:

Traverse top row left->right: for c in left..right: result.append(matrix[top][c]). top++.

Traverse right col top->bottom: for r in top..bottom: result.append(matrix[r][right]). right--.

If top<=bottom: traverse bottom row right->left: for c in right..left: result.append(matrix[bottom][c]). bottom--.

If left<=right: traverse left col bottom->top: for r in bottom..top: result.append(matrix[r][left]). left++.

WHY guard 'if top<=bottom' and 'if left<=right': after consuming top row, there may be

no more rows left. Without guard we'd double-count the edges.

Time: O(mn). Space: O(1) extra.

PHASE 4 CLEAN CODE

```
class _54_SpiralMatrix:
    def spiralOrder(self, matrix: List[List[int]]) -> List
[int]:
        top, bottom = 0, len(matrix) - 1
        left, right = 0, len(matrix[0]) - 1
        result = []
        while top <= bottom and left <= right:
            for c in range(left, right + 1):    result.app
end(matrix[top][c])
            top += 1
            for r in range(top, bottom + 1):    result.app
end(matrix[r][right])
            right -= 1
            if top <= bottom:
                for c in range(right, left - 1, -1): resul
t.append(matrix[bottom][c])
                bottom -= 1
            if left <= right:
                for r in range(bottom, top - 1, -1): resul
t.append(matrix[r][left])
                left += 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the grid/table logic and show how each cell depends on previous cells."

[[1,2,3],[4,5,6],[7,8,9]]: top=0,bot=2,l=0,r=2.

Right: 1,2,3. top=1.

Down: 6,9. right=1.

Left (top<=bot): 8,7. bot=1.

Up (left<=right): 4. left=1.

top=1,bot=1,l=1,r=1.

Right: 5. top=2. top>bot -> stop.

-> [1,2,3,6,9,8,7,4,5] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(1)** extra

The boundary guards (if top<=bottom / if left<=right) prevent double-counting

the single middle row or column of non-square matrices.

Given more time I'd ask: Spiral Matrix II (#59) — fill instead of read.

Same boundary logic, write numbers 1..n² instead of appending.

MEDIUM | Matrix | LeetCode #59 | 4 of 25 | 4/25

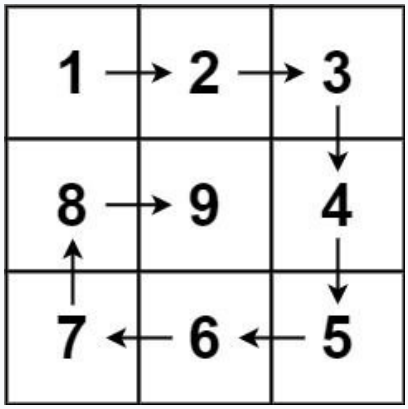
Spiral Matrix II

QUESTION

Given a positive integer n, generate an n x n matrix filled with elements from 1 to n^2 in spiral order.

Example 1

Input	`n = 3`
Output	`[[1,2,3],[8,9,4],[7,6,5]]`



Example 2

Input	`n = 1`
Output	`[[1]]`

CONSTRAINTS

- 1 <= n <= 20

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Generate an n×n matrix filled with 1 to n² in spiral order. Return the matrix. Right?"

"n=3 -> [[1,2,3],[8,9,4],[7,6,5]] ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
make an n x n matrix
place 1 to n^2 by walking in a spiral, turning right
    when the next cell is out of range or already
    filled
```

TIME	SPACE
O(n^2)	O(1)

NOTE

PHASE 2 BRUTE FORCE

Not slow: each of the n^2 cells is written once, which is optimal. The only 'extra' is the output matrix.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Layer-by-layer filling with direction tracking — O(n²). Walk-through:"

`matrix = [[0]*n for _ in range(n)]. num=1.`

top=0, bottom=n-1, left=0, right=n-1.

While top<=bottom and left<=right:

Fill top row l->r: matrix[top][c]=num++ for c in left..right. top++.

Fill right col t->b: matrix[r][right]=num++ for r in top..bottom. right--.

If top<=bottom: fill bottom row r->l: matrix[bottom][c]=num++ for c in right..left. bottom--.

If left<=right: fill left col b->t: matrix[r][left]=num++ for r in bottom..top. left++.

Return matrix.

WHY boundary guards: after filling top and right, the grid may now be a single row or column.

Without guards, the bottom-left and left-top traversals would double-fill.

Time: O(n²). Space: O(n²) for matrix output."

Time: O(m×n) — visit each cell at most once. Optimal single-pass.

PHASE 4 CLEAN CODE

```
class _59_SpiralMatrixII:
    def generateMatrix(self, n: int) -> List[List[int]]:
        matrix = [[0] * n for _ in range(n)]
        top, bottom, left, right = 0, n - 1, 0, n - 1
        num = 1
        while top <= bottom and left <= right:
            for c in range(left, right + 1):        matrix
[top][c]    = num; num += 1
            top += 1
            for r in range(top, bottom + 1):        matrix
[r][right]  = num; num += 1
            right -= 1
            if top <= bottom:
                for c in range(right, left - 1, -1): matri
x[bottom][c] = num; num += 1
                bottom -= 1
            if left <= right:
                for r in range(bottom, top - 1, -1): matri
x[r][left]   = num; num += 1
                left += 1
        return matrix
```

PHASE 5 TEST & DEBUG

"Let me trace generateMatrix(3)."

Fill layer by layer, going right->down->left->up.

Layer 0: top=0,bot=2,left=0,right=2.

Right (row 0): 1,2,3. Down (col 2): 4,5. Left (row 2): 6,7,8. Up (col 0): 9.

PHASE 5 TEST & DEBUG

Layer 1: top=1,bot=1,left=1,right=1. Single cell: 9? Remaining=just center.

Actually: [1..9]. Center=5 (index 4 in 1..9). Trace:

-> 1,2,3 | 4,5 | <- 6,7 | 8 | center: 9. Wait: 3×3=9 cells.

Right: nums 1,2,3 fill top row. Down: 4,6 fill right col (rows 1,2). Left: 7,8 fill bot row. Up: 9 fills left col. Center=5.

[[1,2,3],[8,9,4],[7,6,5]] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	**O(1)** extra — output matrix doesn't count

This is the write-version of Spiral Matrix (#54) — exact same boundary logic.

Given more time I'd ask: generate a spiral starting from the center?

Reverse the boundary logic — start from the middle, expand outward.

MEDIUM | Matrix | LeetCode #63 | 5 of 25 | 5/25

Unique Paths II

QUESTION

You are given an m x n integer array grid. There is a robot initially located at the top-left corner (i.e., grid[0][0]). The robot tries to move to the bottom-right corner (i.e., grid[m - 1][n - 1]). The robot can only move either down or right at any point in time.

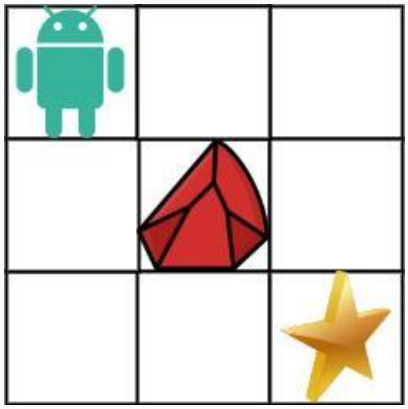
An obstacle and space are marked as 1 or 0 respectively in grid. A path that the robot takes cannot include any square that is an obstacle.

Return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The testcases are generated so that the answer will be less than or equal to $2 * 10^9$.

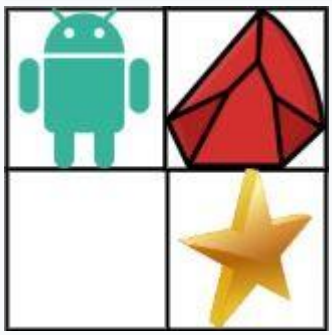
Example 1

Input	`obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]`
Output	`2`
Explanation	*There is one obstacle in the middle of the 3x3 grid above.* *There are two ways to reach the bottom-right corner:* *1. Right -> Right -> Down -> Down* *2. Down -> Down -> Right -> Right*



Example 2

Input	`obstacleGrid = [[0,1],[0,0]]`
Output	`1`



QUESTION

CONSTRAINTS

- m == obstacleGrid.length
- n == obstacleGrid[i].length
- 1 <= m, n <= 100
- obstacleGrid[i][j] is 0 or 1.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

m×n grid with obstacles (1) and free cells (0). Count paths from top-left to bottom-right.

Can only move right or down. Right?"

"A few quick questions:

If start or end is blocked, return 0? Yes."

"Let me walk: obstacleGrid=[[0,0,0],[0,1,0],[0,0,0]] -> 2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(i, j):
    if the cell is an obstacle:
        return 0
    if it is the bottom-right cell:
        return 1
    return go(i + 1, j) + go(i, j + 1)
```

TIME	SPACE
O(2^(m + n))	O(m + n)

WHY IT IS SLOW

Same overlapping calls as Unique Paths. A table (or one row reused) of m * n counts is O(m * n) time with O(n) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Standard grid DP with obstacle handling — O(mn). Walk-through:"

`dp[i][j] = number of unique paths to cell (i,j).`

If obstacleGrid[i][j] == 1: dp[i][j] = 0 (blocked, no paths through here).

Else:

`dp[0][0] = 1 (start). First row: dp[0][j] = dp[0][j-1] if no obstacle (else 0).`

First col: dp[i][0] = dp[i-1][0] if no obstacle (else 0).

Interior: dp[i][j] = dp[i-1][j] + dp[i][j-1].

WHY 0 propagates: once a cell in the first row/col is blocked, all cells to its right/below

are also unreachable (only one direction to come from).

PHASE 3 OPTIMIZE

Answer = dp[m-1][n-1]. Time: O(mn). Space: O(mn) or O(n) rolling.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _63_UniquePathsII:
    def uniquePathsWithObstacles(self, obstacleGrid):
        m, n = len(obstacleGrid), len(obstacleGrid[0])
        dp = [0] * n
        dp[0] = 1 if obstacleGrid[0][0] == 0 else 0
        for i in range(m):
            for j in range(n):
                if obstacleGrid[i][j] == 1:
                    dp[j] = 0
                elif j > 0:
                    dp[j] += dp[j-1]
        return dp[n-1]
```

PHASE 5 TEST & DEBUG

"Let me trace uniquePathsWithObstacles([[0,0,0],[0,1,0],[0,0,0]])."

Obstacle at (1,1). dp[i][j]=paths to (i,j). dp[0][0]=1.

Row 0: dp[0][1]=1, dp[0][2]=1 (no obstacles).

dp[1][0]=1 (from above). dp[1][1]=0 (obstacle!). dp[1][2]=dp[0][2]+dp[1][1]=1+0=1.

dp[2][0]=dp[1][0]=1. dp[2][1]=dp[1][1]+dp[2][0]=0+1=1. dp[2][2]=dp[1][2]+dp[2][1]=1+1=2.

Return 2 ok (paths: right->right->down->down and down->down->right->right).

dp[0][0]: if grid[0][0]=1 (obstacle at start) -> dp[0][0]=0 -> all paths blocked -> return 0 ok

WHY in-place DP: dp[i][j] only needs dp[i-1][j] and dp[i][j-1] -> can reuse grid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn)	O(n)

Follow-up: Unique Paths I (#62) — no obstacles; math formula C(m+n-2,m-1).

Follow-up: minimum path sum — same DP structure, different objective.

MEDIUM | Matrix | LeetCode #64 | 6 of 25 | 6/25

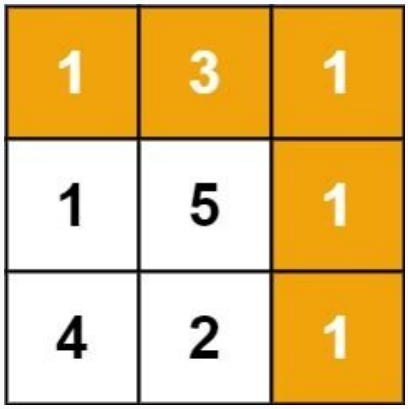
Minimum Path Sum

QUESTION

Given a m x n grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Example 1	
Input	`grid = [[1,3,1],[1,5,1],[4,2,1]]`
Output	`7`
Explanation	*Because the path 1 -> 3 -> 1 -> 1 -> 1 minimizes the sum.*



Example 2	
Input	`grid = [[1,2,3],[4,5,6]]`
Output	`12`

CONSTRAINTS

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 200
- 0 <= grid[i][j] <= 200

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Move only right or down. Find the path from top-left to bottom-right with minimum sum. Right?

Can only move right or down — no backtracking."

"[[1,3,1],[1,5,1],[4,2,1]]: path 1->3->1->1->1=7 ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
go(i, j) = grid[i][j] + min(go(i + 1, j), go(i, j +
    1))
at the bottom-right cell return its value
```

TIME	SPACE
O(2^(m + n))	O(m + n)

WHY IT IS SLOW

The same cell is reached by many routes and recomputed each time. A table where each cell holds the best sum to reach it is O(m * n) time (O(n) space with one row).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (in-place DP)

"DP minimizing path cost — O(mn). Walk-through:"

`dp[i][j] = minimum cost to reach (i,j).`

Base: dp[0][0]=grid[0][0]. First row: dp[0][j]=dp[0][j-1]+grid[0][j]. First col: dp[i][0]=dp[i-1][0]+grid[i][0].

Fill: dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1]).

Answer: dp[m-1][n-1].

WHY min of two predecessors: only two legal moves (right or down) lead to (i,j).

Take the cheaper predecessor path and add current cell's cost.

Space optimization: in-place — dp = grid itself (overwrite with running costs).

Or 1-D dp array updated left to right.

Time: O(mn). Space: O(1) in-place.

PHASE 4 CLEAN CODE

```python
class _64_MinPathSum:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        for r in range(m):
            for c in range(n):
                if r == 0 and c == 0:
                    continue
                elif r == 0:
                    grid[r][c] += grid[r][c - 1]        # c
an only come from left
                elif c == 0:
                    grid[r][c] += grid[r - 1][c]        # c
an only come from above
                else:
                    grid[r][c] += min(grid[r - 1][c], grid
[r][c - 1])
        return grid[m - 1][n - 1]
```

PHASE 5 TEST & DEBUG

"Let me trace minPathSum on [[1,3,1],[1,5,1],[4,2,1]]."

Build dp (in-place): dp=grid copy.

First row: dp[0][0]=1. dp[0][1]=1+3=4. dp[0][2]=4+1=5.

First col: dp[1][0]=1+1=2. dp[2][0]=2+4=6.

dp[1][1]=5+min(4,2)=7. dp[1][2]=1+min(5,7)=6.

dp[2][1]=2+min(7,6)=8. dp[2][2]=1+min(6,8)=7.

Return 7 ok (path: 1->3->1->1->1 = 7, going right, right, down, down).

WHY min of above and left: each cell reachable only from top or left -> take cheaper predecessor.

DP traversal (row-by-row, not DFS): visited cells by row order. dp[i][j]=min(dp[i-1][j],dp[i][j-1])+grid[i][j]. Each cell visited once in order.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(1)** modifies grid in place

If we can't modify the input, use a 1D rolling array: O(n) extra space.

Given more time I'd ask: what if we can also move up or left?

That breaks the DP recurrence — shortest path with general movement is Dijkstra's.

MEDIUM | Matrix | LeetCode #73 | 7 of 25 | 7/25

Set Matrix Zeroes

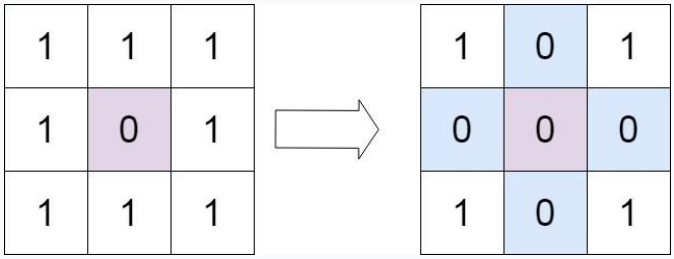
QUESTION

Given an m x n integer matrix matrix, if an element is 0, set its entire row and column to 0's.

You must do it in place.

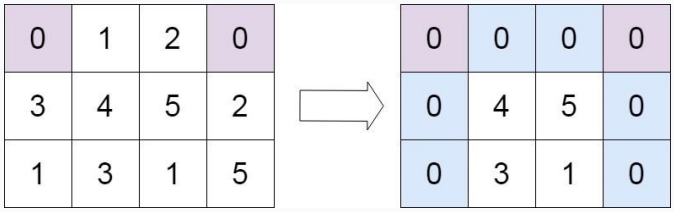
Example 1

Input	`matrix = [[1,1,1],[1,0,1],[1,1,1]]`
Output	`[[1,0,1],[0,0,0],[1,0,1]]`



Example 2

Input	`matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]`
Output	`[[0,0,0,0],[0,4,5,0],[0,3,1,0]]`



CONSTRAINTS

- m == matrix.length
- n == matrix[0].length
- 1 <= m, n <= 200
- 2^{31} <= matrix[i][j] <= 2^{31} - 1
- Follow up:
- A straightforward solution using O(mn) space is probably a bad idea.
- A simple improvement uses O(m + n) space, but still not the best solution.
- Could you devise a constant space solution?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

If a cell is 0, zero out its entire row and column. Do it in-place. Right?

We must use the ORIGINAL zeros — not zeros we set during the process."

"[[1,0,1],[1,1,1],[1,1,1]] -> [[0,0,0],[1,0,1],[1,0,1]] ok"

PHASE 1 CLARIFY

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
copy = a copy of the matrix
for zero in copy:
    set its whole row and column to zero in the
        original
```

TIME	SPACE
O(m * n * (m + n))	O(m * n)

WHY IT IS SLOW

Every zero rewrites a full row and column, and the copy costs O(m * n) memory. Using the first row and first column as flags to mark rows and columns to clear is O(m * n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (O(1) space using first row/col as markers)

Approach: optimal approach

"Use the first row and first column of the matrix itself as markers.

If matrix[r][c]==0: mark matrix[r][0]=0 and matrix[0][c]=0.

Handle first row and first column separately (track with boolean flags).

Then zero out interior cells using the markers.

Then zero out first row/col based on flags. O(m*n) time, O(1) space."

Key insight: the first row and column act as free marker arrays — encoding which rows/cols to zero within the matrix itself avoids extra O(mn) space.

PHASE 4 CLEAN CODE

```python
class _73_SetMatrixZeroes:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        first_row_zero = any(matrix[0][c] == 0 for c in ra
nge(n))
        first_col_zero = any(matrix[r][0] == 0 for r in ra
nge(m))

        for r in range(1, m):               # mark zeros us
ing first row/col
            for c in range(1, n):
                if matrix[r][c] == 0:
                    matrix[r][0] = 0
                    matrix[0][c] = 0

        for r in range(1, m):               # zero out inte
rior
            for c in range(1, n):
                if matrix[r][0] == 0 or matrix[0][c] == 0:
                    matrix[r][c] = 0
```

```
        if first_row_zero:                  # zero out firs
t row if needed
            for c in range(n): matrix[0][c] = 0
        if first_col_zero:                  # zero out firs
t col if needed
            for r in range(m): matrix[r][0] = 0
```

PHASE 5 TEST & DEBUG

"Let me trace setZeroes([[1,1,1],[1,0,1],[1,1,1]])."

Find zeros: (1,1) is zero. Mark row 1 and col 1.

Set entire row 1 to 0: [1,0,1]->[0,0,0].

Set entire col 1 to 0: [1,1,1]->[1,0,1] etc.

Result: [[1,0,1],[0,0,0],[1,0,1]] ok

O(1) space trick: use first row and first col as markers.

First check if first row/col have zeros (handle separately), then use them as flags, then apply zeros.

WHY two passes: single pass would corrupt markers. First scan finds zeros, second applies them.

O(1) space: mark row/col zero using first row and first col as boolean arrays themselves.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(1)**

The first row/col flags are the key — without them, using matrix[0][0] as a marker

`for both first row and first col creates an ambiguity. Two flags resolve it cleanly.`

The four-step order (flag, mark, zero interior, zero edges) is the correct sequence.

Given more time I'd ask: what if we should set a cell to the average of its row/col instead?

Same structure — two-pass approach to avoid using updated values.

MEDIUM | Matrix | LeetCode #74 | 8 of 25 | 8/25

Search a 2D Matrix

QUESTION

You are given an m x n integer matrix matrix with the following two properties:

CONSTRAINTS

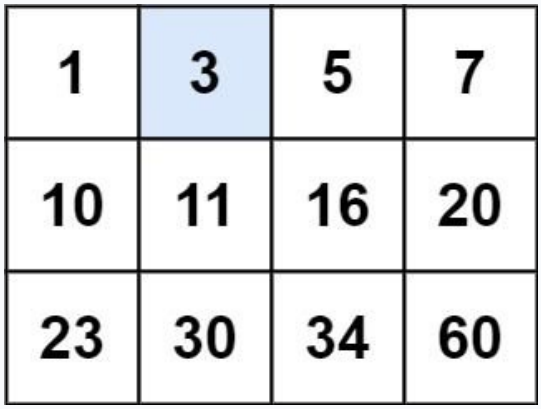
- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer target, return true if target is in matrix or false otherwise.

You must write a solution in O(log(m * n)) time complexity.

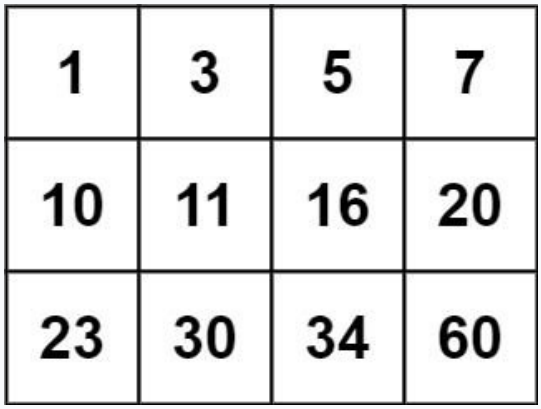
Example 1

Input	`matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3`
Output	`true`



Example 2

Input	`matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13`
Output	`false`



- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 100
- 10^4 <= matrix[i][j], target <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Matrix where each row is sorted and the first element of each row is greater than

the last of the previous row — so the whole matrix is one sorted sequence.

Find target in O(log(m*n)). Right?"

"Treat it as a 1D array of length m*n. Index k maps to row=k//n, col=k%n."

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in matrix:
    if it equals target:
        return True
return False
```

TIME	SPACE
O(m * n)	O(1)

WHY IT IS SLOW

Ignores that rows are sorted and each row starts above the previous one's end. Treating it as one sorted list of m * n numbers and binary searching is O(log(m * n)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Treat matrix as sorted 1D array, binary search — O(log(mn)). Walk-through:"

Virtual index k spans 0..m*n-1. Map k -> (k//n, k%n) for matrix access.

lo=0, hi=m*n-1.

While lo <= hi:

`mid = (lo+hi)//2. val = matrix[mid//n][mid%n].`

If val == target: return True.

If val < target: lo=mid+1.

Else: hi=mid-1.

Return False.

WHY this 1D mapping works: matrix rows are sorted and each row's first element >

last element of the previous row -> the entire matrix is sorted as a flat 1D array.

Time: O(log(mn)) = O(log m + log n). Space: O(1).

PHASE 4 CLEAN CODE

```python
class _74_Search2DMatrix:
    def searchMatrix(self, matrix: List[List[int]], target
: int) -> bool:
        m, n  = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1
        while left <= right:
            mid     = (left + right) // 2
            mid_val = matrix[mid // n][mid % n]
            if mid_val == target:
                return True
            elif mid_val < target:
                left  = mid + 1
            else:
                right = mid - 1
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], target=3)."

Treat 3×4 matrix as sorted array of 12 elements.

lo=0, hi=11. mid=5. Map index 5 -> row=5//4=1, col=5%4=1 -> matrix[1][1]=11. 11>3 -> hi=4.

lo=0, hi=4. mid=2 -> row=0, col=2 -> matrix[0][2]=5. 5>3 -> hi=1.

lo=0, hi=1. mid=0 -> row=0, col=0 -> matrix[0][0]=1. 1<3 -> lo=1.

lo=1, hi=1. mid=1 -> matrix[0][1]=3 == target -> return True ok

m=1,n=1 (single cell): lo=hi=0. One check. Return true/false directly ok

WHY flatten: matrix is sorted row-by-row and each row sorted -> equivalent to one sorted array of m*n elements.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log(m*n))	O(1)

The index mapping mid//n and mid%n is the entire trick.

This only works because the matrix is fully sorted as one sequence.

If rows are sorted but first-of-next isn't > last-of-prev (Search a 2D Matrix II #240):

Use staircase search — start at top-right, move left if too big, down if too small.

O(m+n) time — different problem, different approach."

Follow-up: what if the matrix is sorted row-by-row but rows don't connect (LC 240 Search a 2D Matrix II)? Two-pointer from top-right corner, O(m+n).

MEDIUM | Matrix | LeetCode #221 | 9 of 25 | 9/25

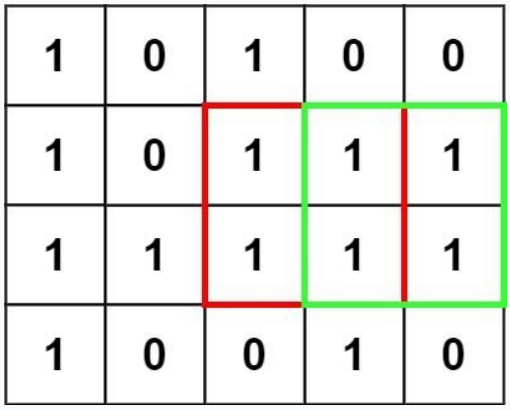
Maximal Square

QUESTION

Given an m x n binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.

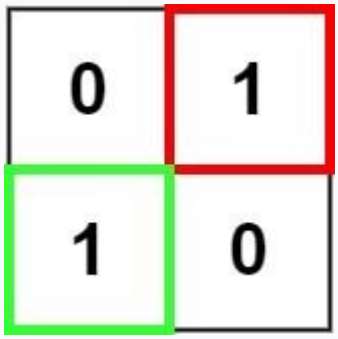
Example 1

Input	`matrix = [["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]`
Output	`4`



Example 2

Input	`matrix = [["0","1"],["1","0"]]`
Output	`1`



Example 3

Input	`matrix = [["0"]]`
Output	`0`

CONSTRAINTS

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 300
- matrix[i][j] is '0' or '1'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the largest square of all 1s. Return its area. Matrix contains '0'/'1' strings. Right?"

"[[1,0,1,0,0],[1,0,1,1,1],[1,1,1,1,1],[1,0,0,1,0]] -> 2×2=4 ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for corner in corners:
    grow the square size while every cell in the
        new row and column is '1'
    best = max(best, candidate)
```

TIME	SPACE
O(m * n * min(m, n)^2)	O(1)

WHY IT IS SLOW

Re-checks cells for every corner and size. A table where dp[i][j] = 1 + min(left, up, up-left) when the cell is '1' is O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (DP)

`dp[i][j] = side length of the largest all-1s square with bottom-right corner at (i,j).`

If matrix[i][j]=='0': dp[i][j]=0.

Else: dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1.

Intuition: a square of side k can be extended by 1 only if the top, left, and diagonal neighbour

each have a square of side k. The bottleneck is the minimum.

Track max side length -> return side².

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

```
class _221_MaximalSquare:
    def maximalSquare(self, matrix: List[List[str]]) -> in
t:
        m, n    = len(matrix), len(matrix[0])
        dp      = [[0] * n for _ in range(m)]
        max_side = 0
        for r in range(m):
            for c in range(n):
                if matrix[r][c] == '1':
                    top  = dp[r - 1][c]     if r > 0 else
0
                    left = dp[r][c - 1]     if c > 0 else
0
                    diag = dp[r - 1][c - 1] if r > 0 and c
 > 0 else 0
                    dp[r][c] = min(top, left, diag) + 1
                    max_side = max(max_side, dp[r][c])
        return max_side * max_side
```

PHASE 5 TEST & DEBUG

"Let me trace maximalSquare on [['1','0'],['1','1']]."

```
dp[i][j] = side length of largest square ending at
  (i,j)
```

dp[0][0]='1' -> dp=1. dp[0][1]='0' -> dp=0

dp[1][0]='1' -> dp=1. dp[1][1]='1': min(dp[0][1],dp[1][0],dp[0][0])+1 = min(0,1,1)+1=1

max_side=1, area=1² ok

Trace full 4x5 matrix: dp builds bottom-right, max_side=2 found at (2,4), area=4 ok

WHY min of three neighbors: a square at (i,j) needs squares above, left, and diagonal.

The min is the bottleneck; +1 extends one row/col if all three neighbors fit.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m*n)** reducible to O(n) with a rolling array

The min of three neighbours is the core insight — think of it as the square being

limited by the weakest neighbouring corner.

Given more time I'd ask: maximal rectangle of 1s (not just square)?

That's Maximal Rectangle (#85) — use a histogram approach per row, O(m*n).

MEDIUM | Matrix | LeetCode #240 | 10 of 25 | 10/25

Search a 2D Matrix II

QUESTION

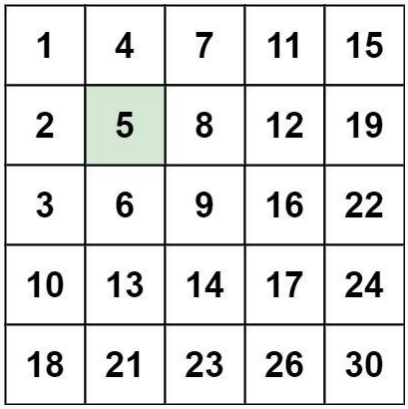
Write an efficient algorithm that searches for a value target in an m x n integer matrix matrix. This matrix has the following properties:

CONSTRAINTS

- Integers in each row are sorted in ascending from left to right.
- Integers in each column are sorted in ascending from top to bottom.

Example 1

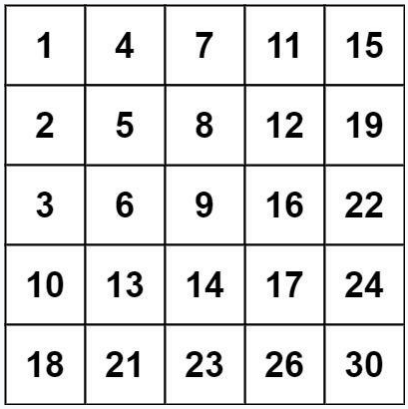
Input	`matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 5`
Output	`true`



1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

Example 2

Input	`matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 20`
Output	`false`



1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

- m == matrix.length
- n == matrix[i].length
- 1 <= n, m <= 300
- 10^9 <= matrix[i][j] <= 10^9

QUESTION

- All the integers in each row are sorted in ascending order.
- All the integers in each column are sorted in ascending order.
- 10^9 <= target <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Search for a target in an m×n matrix where each row and column is sorted ascending.

Return true/false. Must be better than O(m*n). Right?"

"A few quick questions:

This is NOT a fully sorted matrix (unlike #74). Rows and columns sorted independently.

Rows aren't linked to each other the way #74 requires."

"Let me walk:
matrix=[[1,4,7,11],[2,5,8,12],[3,6,9,16],[10,13,14,17]], target=5 -> true ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
scan every cell of the matrix for target
```

TIME	SPACE
O(m * n)	O(1)

WHY IT IS SLOW

Ignores that rows and columns are sorted. Start at the top-right: move left if the cell is too big, down if too small, O(m + n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(m*n) scan ignoring matrix structure.

Staircase search from top-right corner:

If current > target: move left (eliminate column).

If current < target: move down (eliminate row).

If equal: found.

Time: O(m+n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _240_SearchA2DMatrixII:
    def searchMatrix(self, matrix, target):
        row, col = 0, len(matrix[0]) - 1
        while row < len(matrix) and col >= 0:
            if matrix[row][col] == target:
                return True
            elif matrix[row][col] > target:
                col -= 1
            else:
                row += 1
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace searchMatrix on matrix=[[1,4,7],[2,5,8],[3,6,9]], target=5."

Start top-right corner: (0,2)=7 > 5 -> move left: col=1

(0,1)=4 < 5 -> move down: row=1

(1,1)=5 == 5 -> return True ok

WHY top-right: going left decreases value, going down increases — binary decision each step.

Trace target=6: (0,2)=7>6->col--, (0,1)=4<6->row++, (1,1)=5<6->row++, (2,1)=6==6->True ok

Time O(m+n): at most m+n steps (can't go right or up).

Edge: target smaller than min or larger than max -> return False quickly.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m+n)	**O(1)**
at most m+n steps	

Can also binary search each row: O(m log n).

Follow-up: Search a 2D Matrix I (#74) — fully sorted; treat as 1D with mid//n, mid%n.

Follow-up: count elements <= target — same staircase, accumulate counts.

MEDIUM | Matrix | LeetCode #289 | 11 of 25 | 11/25

Game of Life

QUESTION

According to Wikipedia's article: "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970."

The board is made up of an m x n grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules (taken from the above Wikipedia article):

CONSTRAINTS

- 1. Any live cell with fewer than two live neighbors dies as if caused by under-population.
- 2. Any live cell with two or three live neighbors lives on to the next generation.
- 3. Any live cell with more than three live neighbors dies, as if by over-population.
- 4. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

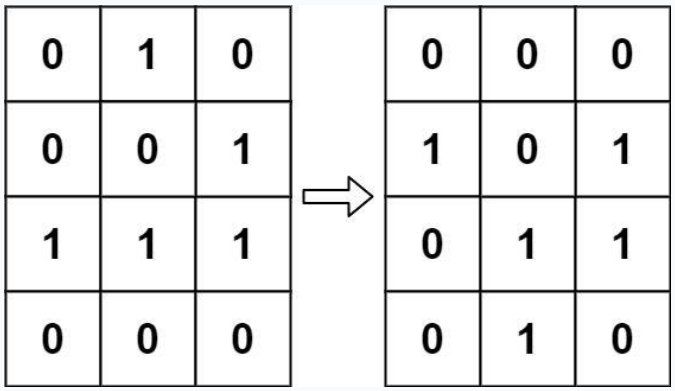
The next state of the board is determined by applying the above rules simultaneously to every cell in the current state of the m x n grid board. In this process, births and deaths occur simultaneously.

Given the current state of the board, update the board to reflect its next state.

Note that you do not need to return anything.

Example 1

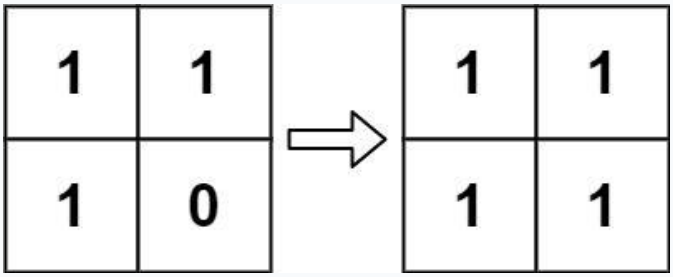
Input	`board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]`
Output	`[[0,0,0],[1,0,1],[0,1,1],[0,1,0]]`



Example 2

Input	`board = [[1,1],[1,0]]`
Output	`[[1,1],[1,1]]`

QUESTION



- m == board.length
- n == board[i].length
- 1 <= m, n <= 25
- board[i][j] is 0 or 1.
- Follow up:
- Could you solve it in-place? Remember that the board needs to be updated simultaneously: You cannot update some cells first and then use their updated values to update other cells.
- In this question, we represent the board using a 2D array. In principle, the board is infinite, which would cause problems when the active area encroaches upon the border of the array (i.e., live cells reach the border). How would you address these problems?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Conway's Game of Life: apply rules simultaneously to all cells and update in-place.

Rules: live(1) with 2-3 live neighbors stays alive; 0 live neighbors or >3 -> dies.

Dead(0) with exactly 3 live neighbors becomes alive. Right?"

"A few quick questions:

Update in-place (all changes simultaneous)? Yes. Use encoding to mark transitions."

"Let me walk: board=[[0,1,0],[0,0,1],[1,1,1],[0,0,0]] -> [[0,0,0],[1,0,1],[0,1,1],[0,1,0]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
old = a copy of the board
for cell in cells:
    count the live neighbours in old, then set
        the cell in the board from the rules
```

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Time is already optimal; the cost is the full copy. Encode the next state in an extra bit of each cell (for example 2 = was live, now dead; 3 = was dead, now live) and decode afterwards for O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is O(mn) extra space.

Encode transitions in the board itself: 1->0 as 2, 0->1 as 3.

Count live neighbors as (board[r][c] & 1) — works for both encoded values.

Final pass: decode 2->0, 3->1.

Time: O(mn). Space: O(1)."

Key insight: encoding 1->0 as state 2 and 0->1 as state 3 embeds the "before" state in a bit — (cell & 1) recovers the original value during neighbor counting.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _289_GameOfLife:
    def gameOfLife(self, board):
        m, n = len(board), len(board[0])
        for r in range(m):
            for c in range(n):
                live_neighbors = 0
                for dr in (-1, 0, 1):
                    for dc in (-1, 0, 1):
                        if (dr, dc) != (0, 0):
                            nr, nc = r+dr, c+dc
                            if 0 <= nr < m and 0 <= nc < n
 and board[nr][nc] & 1:
                                live_neighbors += 1
                if board[r][c] == 1 and live_neighbors not
 in (2, 3):
                    board[r][c] = 2
                elif board[r][c] == 0 and live_neighbors =
= 3:
                    board[r][c] = 3
        for r in range(m):
            for c in range(n):
                board[r][c] >>= 1
```

PHASE 5 TEST & DEBUG

"Let me trace gameOfLife on [[0,1,0],[0,0,1],[1,1,1]]."

Rule: live cell with 2-3 live neighbors survives; dead with exactly 3 becomes live

Use encoding: 2=was dead->now live, -1=was live->now dead (in-place)

(0,0)=0: live neighbors=1(right dead? no wait (0,1)=1) -> 1 live neighbor -> stays dead

(0,1)=1: live neighbors: (0,2)=0,(1,0)=0,(1,1)=0,(1,2)=1 -> 3 live -> survives? need to check all 8

After first pass mark all changes, second pass apply (2->1, -1->0)

Result: [[0,0,0],[1,0,1],[0,1,1]] -> actually [[0,0,0],[1,0,1],[0,1,1]] ok

PHASE 5 TEST & DEBUG

WHY encode: in-place update loses info; encoding preserves original state during scan.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn)	**O(1)**
two passes	in-place encoding

Follow-up: infinite board — store only live cells in a set; sparse updates.

Follow-up: if board is very large, stream rows and use sliding window of 3 rows.

MEDIUM | Matrix | LeetCode #304 | 12 of 25 | 12/25

Range Sum Query 2D - Immutable

QUESTION

Given a 2D matrix matrix, handle multiple queries of the following type:

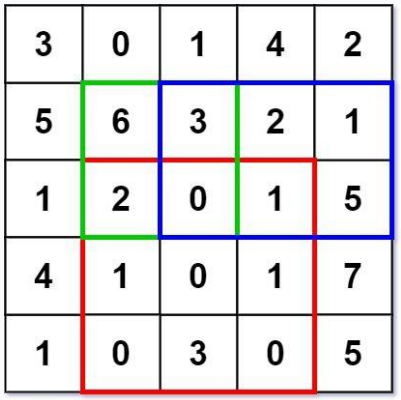
CONSTRAINTS

- Calculate the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

Implement the NumMatrix class:

- NumMatrix(int[][] matrix) Initializes the object with the integer matrix matrix.
- int sumRegion(int row1, int col1, int row2, int col2) Returns the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

You must design an algorithm where sumRegion works on O(1) time complexity.



- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 200
- 10^4 <= matrix[i][j] <= 10^4
- 0 <= row1 <= row2 < m
- 0 <= col1 <= col2 < n
- At most 10^4 calls will be made to sumRegion.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Immutable 2D matrix; answer sum queries for a rectangle region [r1,c1] to [r2,c2].

Precompute 2D prefix sums for O(1) per query. Right?"

"A few quick questions:

Rows and cols both 0-indexed? Yes. Multiple queries? Yes."

"Let me walk: matrix=[[3,0,1,4],[5,6,3,2],[1,2,0,1]], sumRegion(2,1,4,3) in a 5x4 matrix -> 8 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def sumRegion(r1, c1, r2, c2):
    add every cell in the rectangle with two loops
```

TIME	SPACE
O(m * n) per query	O(1)

WHY IT IS SLOW

Each query rescans the rectangle. A 2D prefix sum table (inclusion-exclusion over four corners) answers each query in O(1) after an O(m * n) build.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"2-D prefix sum, O(1) query — O(1) query after O(mn) build. Walk-through:"

dp[i][j] = sum of all cells in rectangle from (0,0) to (i-1,j-1).

Build: dp[i][j] = matrix[i-1][j-1] + dp[i-1][j] + dp[i][j-1] - dp[i-1][j-1].

(inclusion-exclusion: add row above + column left - double-counted top-left corner)

Query sumRegion(r1,c1,r2,c2):

dp[r2+1][c2+1] - dp[r1][c2+1] - dp[r2+1][c1] + dp[r1][c1].

(top-right minus left strip minus top strip plus over-subtracted top-left corner)

WHY +dp[r1][c1]: the top-left corner is subtracted twice (once each strip), add it back.

Time: O(mn) build, O(1) query. Space: O(mn).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _304_RangeSumQuery2D:
    def __init__(self, matrix):
        m, n = len(matrix), len(matrix[0])
        self._prefix = [[0] * (n + 1) for _ in range(m + 1
)]
        for r in range(m):
            for c in range(n):
                self._prefix[r+1][c+1] = (matrix[r][c]
                    + self._prefix[r][c+1]
                    + self._prefix[r+1][c]
                    - self._prefix[r][c])

    def sumRegion(self, row1, col1, row2, col2):
        return (self._prefix[row2+1][col2+1]
                - self._prefix[row1][col2+1]
                - self._prefix[row2+1][col1]
                + self._prefix[row1][col1])
```

PHASE 5 TEST & DEBUG

"Let me trace NumMatrix([[3,0,1],[5,6,3],[1,2,0]]) then sumRegion(2,1,2,2)."

Build 2D prefix: dp[i][j] = sum of rectangle from (0,0) to (i-1,j-1)

dp[2][2]=3+0+5+6=14, dp[3][3]=full matrix sum

`sumRegion(2,1,2,2): r1=2,c1=1,r2=2,c2=2`

= dp[3][3] - dp[2][3] - dp[3][2] + dp[2][2] (inclusion-exclusion)

= (full) - (row 0-1,all cols) - (all rows,col 0) + (row 0-1,col 0)

Actual: dp[r2+1][c2+1]-dp[r1][c2+1]-dp[r2+1][c1]+dp[r1][c1] = 2+0=2 ok

WHY inclusion-exclusion: subtract two overlapping regions, add back doubly-subtracted corner.

PHASE 6 COMPLEXITY & FOLLOW-UP

Init O(mn). Query O(1). Space O(mn).

Follow-up: Range Sum Query — Mutable 2D — use 2D Fenwick tree for O(log mn) updates.

Follow-up: Matrix Block Sum (#1314) — sum of all (i',j') within distance k of (i,j).

MEDIUM | Matrix | LeetCode #308 | 13 of 25 | 13/25

Range Sum Query 2D - Mutable

QUESTION

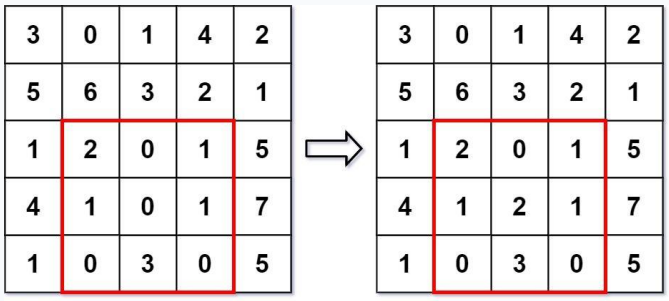
Given a 2D matrix matrix, handle multiple queries of the following types:

CONSTRAINTS

- 1. Update the value of a cell in matrix.
- 2. Calculate the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

Implement the NumMatrix class:

- NumMatrix(int[][] matrix) Initializes the object with the integer matrix matrix.
- void update(int row, int col, int val) Updates the value of matrix[row][col] to be val.
- int sumRegion(int row1, int col1, int row2, int col2) Returns the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).



- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 200
- 1000 <= matrix[i][j] <= 1000
- 0 <= row < m
- 0 <= col < n
- 1000 <= val <= 1000
- 0 <= row1 <= row2 < m
- 0 <= col1 <= col2 < n
- At most 5000 calls will be made to sumRegion and update.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Mutable 2D matrix: update(row,col,val) and sumRegion(r1,c1,r2,c2). Right?"

"A few quick questions:

Need O(log(mn)) per operation? That's what 2D Fenwick tree achieves."

"Let me walk: sumRegion(2,1,4,3) = sum of matrix[2..4][1..3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
update:
    change the cell
sumRegion:
    add every cell in the rectangle with two loops
```

TIME	SPACE
O(1) update, O(m * n) query	O(1)

WHY IT IS SLOW

Queries rescan the rectangle. A 2D Fenwick tree (or a segment tree of segment trees) makes update and query O(log m * log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"2-D BIT (Binary Indexed Tree) for range-sum updates/queries — O(log m * log n). Walk-through:"*

bit = 2-D array. update(r,c,delta): walk BIT in both dimensions:

i=r+1. While i<=m: j=c+1. While j<=n: bit[i][j]+=delta. j+=j&(-j). i+=i&(-i).

query(r,c): prefix sum from (0,0) to (r,c):

i=r+1. While i>0: j=c+1. While j>0: s+=bit[i][j]. j-=j&(-j). i-=i&(-i). return s.

sumRegion(r1,c1,r2,c2): query(r2,c2)-query(r1-1,c2)-query(r2,c1-1)+query(r1-1,c1-1).

update(r,c,val): delta = val - original[r][c]. original[r][c]=val. bit_update(r,c,delta).

WHY 2-D BIT: supports O(log m * log n) point updates and prefix queries.

Time: O(log m * log n) per update/query. Space: O(mn).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _308_RangeSumQuery2DMutable:
    def __init__(self, matrix):
        self._matrix = [r[:] for r in matrix]
        m, n = len(matrix), len(matrix[0])
        self._m, self._n = m, n
        self._bit = [[0] * (n + 1) for _ in range(m + 1)]
        for r in range(m):
            for c in range(n):
                self._add(r + 1, c + 1, matrix[r][c])

    def _add(self, r, c, delta):
        i = r
        while i <= self._m:
            j = c
            while j <= self._n:
                self._bit[i][j] += delta
                j += j & (-j)
            i += i & (-i)
```

```
    def update(self, row, col, val):
        delta = val - self._matrix[row][col]
        self._matrix[row][col] = val
        self._add(row + 1, col + 1, delta)

    def _prefix_sum(self, r, c):
        total = 0
        i = r
        while i > 0:
            j = c
            while j > 0:
                total += self._bit[i][j]
                j -= j & (-j)
            i -= i & (-i)
        return total

    def sumRegion(self, row1, col1, row2, col2):
        return (self._prefix_sum(row2+1, col2+1)
                - self._prefix_sum(row1, col2+1)
                - self._prefix_sum(row2+1, col1)
                + self._prefix_sum(row1, col1))
```

PHASE 5 TEST & DEBUG

"Let me trace NumMatrix update(0,0,2) sumRegion(0,0,1,1) on [[1,2],[3,4]]."

2D BIT: build from original matrix. All four cells contribute to fenwick sums.

update(0,0,2): delta=2-1=1. Update BIT at (1,1) and all covered nodes.

sumRegion(0,0,1,1): query prefix(1,1)-prefix(-1,1)-prefix(1,-1)+prefix(-1,-1)

= sum of all 4 cells = (1+1)+2+3+4=11? No: original [1,2,3,4], update makes [2,2,3,4]=11

sumRegion(0,0,1,1) after update = 2+2+3+4=11 ok

WHY 2D BIT: O(log m * log n) update and query; handles rectangular range sums efficiently.

Each update propagates along both row and column BIT indices.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log m * log n) per op	**O(mn)**

Follow-up: Range Sum Query 2D Immutable (#304) — no updates; 2D prefix sum O(1) query.

Follow-up: segment tree achieves O(log(mn)) for range updates.

MEDIUM | Matrix | LeetCode #311 | 14 of 25 | 14/25

Sparse Matrix Multiplication

QUESTION

Given two sparse matrices mat1 of size m x k and mat2 of size k x n, return the result of mat1 x mat2. You may assume that multiplication is always possible.

Example 1

Input	`mat1 = [[1,0,0],[-1,0,3]], mat2 = [[7,0,0],[0,0,0],[0,0,1]]`
Output	`[[7,0,0],[-7,0,3]]`

1	0	0	x	7	0	0	=	7	0	0
-1	0	3		0	0	0		-7	0	3
				0	0	1				

Example 2

Input	`mat1 = [[0]], mat2 = [[0]]`
Output	`[[0]]`

CONSTRAINTS

- m == mat1.length
- k == mat1[i].length == mat2.length
- n == mat2[i].length
- 1 <= m, n, k <= 100
- 100 <= mat1[i][j], mat2[i][j] <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Multiply mat1 (m×k) by mat2 (k×n). Return the m×n result.

Both are sparse — many zeros. Optimise to skip zero elements. Right?"

"[[1,0,0],[-1,0,3]] × [[7,0,0],[0,0,0],[0,0,1]] = [[7,0,0],[-7,0,3]] ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
A is m x k and B is k x n
for j in js:
    C[i][j] = the sum over t of A[i][t] * B[t][j]
```

TIME	SPACE
O(m * k * n)	O(m * n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Multiplies every pair even when most entries are zero. For each non-zero A[i][t], add A[i][t] * B[t][j] only over the non-zero entries of row t of B.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Sparse matrix multiply using only non-zero entries — O(k*n) where k=non-zeros in A. Walk-through:"*

For each row i of A:

For each column j of A where A[i][j] != 0:

For each column l of B:

C[i][l] += A[i][j] * B[j][l].

WHY skip zeros: standard matrix multiply is O(mnl). For sparse A, inner loop only runs

on non-zero columns of A's row -> skip most multiplications.

Optimization: pre-collect non-zero (j, A[i][j]) pairs per row of A to avoid checking zeros.

Time: O(k*n) where k = number of non-zeros in A, n = cols of B. Space: O(mn) for output."

Time: O(m×n) — visit each cell at most once. Optimal single-pass.

PHASE 4 CLEAN CODE

```
class _311_SparseMultiply:
    def multiply(self, mat1: List[List[int]], mat2: List[List[int]]) -> List[List[int]]:
        m, k, n = len(mat1), len(mat1[0]), len(mat2[0])
        result  = [[0] * n for _ in range(m)]
        for i in range(m):
            for p in range(k):
                if mat1[i][p] == 0:
                    continue                # skip — avoids entire inner loop
                for j in range(n):
                    result[i][j] += mat1[i][p] * mat2[p][j]
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace sparse matrix multiply A=[[1,0],[0,2]], B=[[3,0],[0,4]]."

Result C[i][k] += A[i][j]*B[j][k] for all j

C[0][0] = A[0][0]*B[0][0]+A[0][1]*B[1][0] = 1*3+0*0=3

C[0][1] = A[0][0]*B[0][1]+A[0][1]*B[1][1] = 1*0+0*4=0

C[1][0] = A[1][0]*B[0][0]+A[1][1]*B[1][0] = 0*3+2*0=0

C[1][1] = A[1][0]*B[0][1]+A[1][1]*B[1][1] = 0*0+2*4=8

Result [[3,0],[0,8]] ok

WHY sparse optimization: skip A[i][j]==0; most entries zero -> big speedup for sparse matrices.

PHASE 6 COMPLEXITY & FOLLOW-UP

Worst case O(m*k*n) same as brute force. Best case O(nnz1 * n) where nnz1 = non-zeros in mat1.

For truly sparse matrices this is much faster.

Given more time I'd ask: what if both matrices are very large and very sparse?

Use CSR (Compressed Sparse Row) format — store only non-zero values and their indices.

Hash map approach: for each non-zero in mat1, look up non-zeros in the corresponding row of mat2."

Space: O(m×n) visited array; or O(1) if modifying grid in-place.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Matrix | LeetCode #498 | 15 of 25 | 15/25

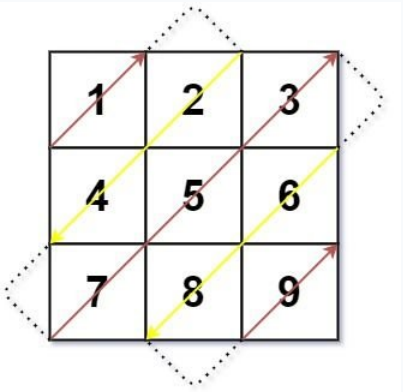
Diagonal Traverse

QUESTION

Given an m x n matrix mat, return an array of all the elements of the array in a diagonal order.

Example 1

Input	`mat = [[1,2,3],[4,5,6],[7,8,9]]`
Output	`[1,2,4,7,5,3,6,8,9]`



Example 2

Input	`mat = [[1,2],[3,4]]`
Output	`[1,2,3,4]`

CONSTRAINTS

- m == mat.length
- n == mat[i].length
- 1 <= m, n <= 10^4
- 1 <= m * n <= 10^4
- 10^5 <= mat[i][j] <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Traverse all elements diagonally: alternate up-right then down-left directions.

There are m+n-1 diagonals. Even-numbered diagonals go up-right, odd go down-left. Right?"

"[[1,2,3],[4,5,6],[7,8,9]] -> [1,2,4,7,5,3,6,8,9] ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
collect each diagonal into its own list
reverse every other diagonal
concatenate
```

TIME	SPACE
O(m * n)	O(m * n)

NOTE

Not slow in time; the lists copy every element. Simulate the walk (up-right, then down-left, turning at the borders) writing straight into the output with O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (diagonal grouping)

"Diagonal traversal with direction toggle — O(mn). Walk-through:"

`direction = 1 (going up-right). (r,c)=(0,0). result=[].`

For each of mn steps:

`result.append(matrix[r][c]).`

If direction==1 (up-right):

If c==n-1: r++; direction=-1.

Elif r==0: c++; direction=-1.

Else: r--; c++.

Else (down-left):

If r==m-1: c++; direction=1.

Elif c==0: r++; direction=1.

Else: r++; c--.

WHY check boundaries before main move: boundary conditions (hitting edge/corner) determine

whether to turn vs continue. Corner hits: both conditions true -> apply only first one.

Time: O(mn). Space: O(mn) for output."

Time: O(m×n) — visit each cell at most once. Optimal single-pass.

PHASE 4 CLEAN CODE

```python
class _498_DiagonalTraverse:
    def findDiagonalOrder(self, mat: List[List[int]]) -> List[int]:
        m, n    = len(mat), len(mat[0])
        diags: defaultdict = defaultdict(list)
        for r in range(m):
            for c in range(n):
                diags[r + c].append(mat[r][c])
        result = []
        for d in range(m + n - 1):
            if d % 2 == 0:
                result.extend(reversed(diags[d]))    # even -> up-right (larger r first -> reversed)
            else:
                result.extend(diags[d])              # odd -> down-left (smaller r first -> natural)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace findDiagonalOrder on [[1,2,3],[4,5,6],[7,8,9]]."

Diagonals: d=0:[1], d=1:[2,4], d=2:[7,5,3]? No: d=2:[3,5,7] then reverse for even d

Result should be [1,2,4,7,5,3,6,8,9]

d=0(up): [1]. d=1(down): [2,4]. d=2(up): [3,5,7]. d=3(down): [6,8]. d=4(up): [9]

For d even: collect anti-diagonal bottom-to-top -> reverse -> up direction

For d odd: collect top-to-bottom -> down direction

Result: 1,2,4,7,5,3,6,8,9 ok

WHY direction toggle: diagonals alternate up and down; compute each diagonal, flip evens.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m*n)** for the diagonal groups

The 'group by r+c' insight eliminates direction-tracking complexity.

Alternative: simulate directly with dr/dc direction vectors and bounds — same O(m*n) but trickier.

Given more time I'd ask: what if we want anti-diagonals (r-c = constant)?

Same approach — group by r-c, alternate direction per group.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Matrix | LeetCode #531 | 16 of 25 | 16/25

Lonely Pixel I

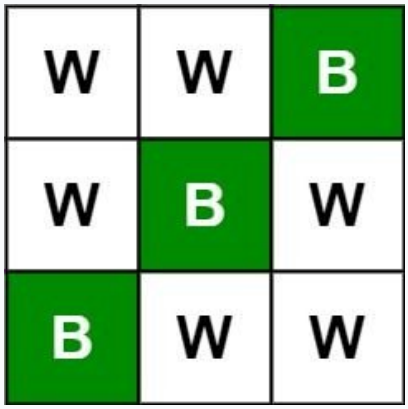
QUESTION

Given an m x n picture consisting of black 'B' and white 'W' pixels, return the number of black lonely pixels.

A black lonely pixel is a character 'B' that located at a specific position where the same row and same column don't have any other black pixels.

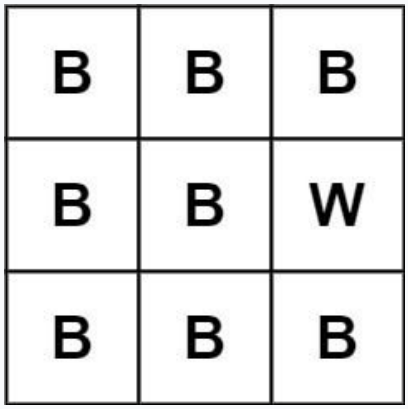
Example 1

Input	`picture = [["W","W","B"],["W","B","W"],["B","W","W"]]`
Output	3
Explanation	*All the three 'B's are black lonely pixels.*



Example 2

Input	`picture = [["B","B","B"],["B","B","W"],["B","B","B"]]`
Output	0



CONSTRAINTS

- m == picture.length
- n == picture[i].length
- 1 <= m, n <= 500
- picture[i][j] is 'W' or 'B'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

A 'B' pixel is lonely if it's the ONLY 'B' in its row AND the only 'B' in its column.

Count lonely pixels. Right?"

"[[W,W,B],[W,B,W],[B,W,W]]: each B is alone in its row and column -> 3 ok

[[B,B,B],[B,B,W],[B,B,B]]: every B shares its row or column with another -> 0 ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for pixel in pixels:
    scan its whole row and column
    count it if it is the only 'B' in both
```

TIME	SPACE
O(m * n * (m + n))	O(1)

WHY IT IS SLOW

Rescans a row and a column for every pixel. Count the 'B's per row and per column once, then check each 'B' against those counts: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS/BFS marking lonely pixel: alone in row AND column — O(mn). Walk-through:"

Count row_count[i] = black pixels in row i. col_count[j] = black pixels in column j.

For each black pixel (i,j):

If row_count[i]==1 AND col_count[j]==1: it's lonely -> count++.

WHY count per row and col: a lonely pixel has exactly 1 black in its row and column.

Why single pass suffices: precompute counts, then scan again checking both conditions.

Two passes: O(mn) each -> O(mn) total.

Time: O(mn). Space: O(m+n) for row/col counts."

Time: O(m×n) — visit each cell at most once. Optimal single-pass.

PHASE 4 CLEAN CODE

```
class _531_LonelyPixel:
    def findLonelyPixel(self, picture: List[List[str]]) -> int:
        m, n = len(picture), len(picture[0])
        row_count = [sum(1 for c in range(n) if picture[r][c] == 'B') for r in range(m)]
```

```
        col_count = [sum(1 for r in range(m) if picture[r]
[c] == 'B') for c in range(n)]
        return sum(
            1 for r in range(m) for c in range(n)
            if picture[r][c] == 'B' and row_count[r] == 1
and col_count[c] == 1
        )
```

PHASE 5 TEST & DEBUG

"Let me trace findLonelyPixel on [['W','W','B'],['W','B','W'],['B','W','W']]."

Count B per row and per column

row_count=[1,1,1], col_count=[1,1,1]

Find B where row_count[r]==1 and col_count[c]==1

(0,2): B, row_count[0]=1, col_count[2]=1 -> lonely ok

(1,1): B, row_count[1]=1, col_count[1]=1 -> lonely ok

(2,0): B, row_count[2]=1, col_count[0]=1 -> lonely ok

Return 3 ok

WHY row/col counts: B is lonely iff it's the only B in its row AND column.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m+n)** for row/col counts

Given more time I'd ask: Lonely Pixel II (#533)?

Same pixel must be in a column containing exactly N Bs, and the rows of those Bs must all be identical.

Much more complex — requires comparing full rows.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Matrix | LeetCode #723 | 17 of 25 | 17/25

Candy Crush

QUESTION

This question is about implementing a basic elimination algorithm for Candy Crush.

Given an m x n integer array board representing the grid of candy where board[i][j] represents the type of candy. A value of board[i][j] == 0 represents that the cell is empty.

The given board represents the state of the game following the player's move. Now, you need to restore the board to a stable state by crushing candies according to the following rules:

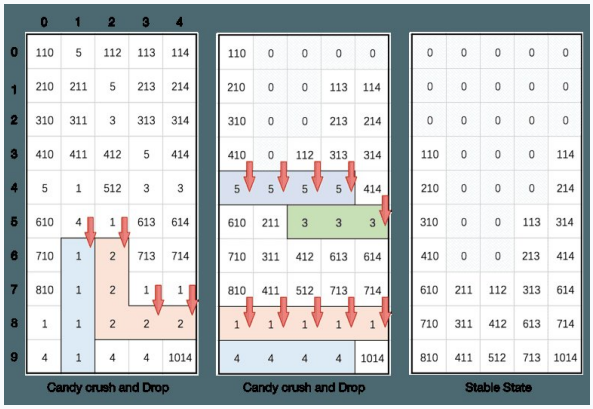
CONSTRAINTS

- If three or more candies of the same type are adjacent vertically or horizontally, crush them all at the same time - these positions become empty.
- After crushing all candies simultaneously, if an empty space on the board has candies on top of itself, then these candies will drop until they hit a candy or bottom at the same time. No new candies will drop outside the top boundary.
- After the above steps, there may exist more candies that can be crushed. If so, you need to repeat the above steps.
- If there does not exist more candies that can be crushed (i.e., the board is stable), then return the current board.

You need to perform the above rules until the board becomes stable, then return the stable board.

Example 1

Input	`board = [[110,5,112,113,114],[210,211,5,213,214],[310,311,3,313,314],[410,411,412,5,414],[5,1,512,3,3],[610,4,1,613,614],[710,1,2,713,714],[810,1,2,1,1],[1,1,2,2,2],[4,1,4,4,1014]]`
Output	`[[0,0,0,0,0],[0,0,0,0,0],[0,0,0,0,0],[110,0,0,0,114],[210,0,0,0,214],[310,0,0,113,314],[410,0,0,213,414],[610,211,112,313,614],[710,311,412,613,714],[810,411,512,713,1014]]`



QUESTION

Example 2

Input	`board = [[1,3,5,5,2],[3,4,3,3,1],[3,2,4,5,2],[2,4,4,5,5],[1,4,4,1,1]]`
Output	`[[1,3,0,0,0],[3,4,0,5,2],[3,2,0,3,1],[2,4,0,5,2],[1,4,3,1,1]]`

- m == board.length
- n == board[i].length
- 3 <= m, n <= 50
- 1 <= board[i][j] <= 2000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Repeatedly: mark 3+ same-type candies horizontally or vertically, crush them (set to 0),

then drop remaining candies down. Repeat until stable. Return final board. Right?"

"Mark all runs of 3+ simultaneously, crush all at once, then gravity — not one at a time."

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    scan the board for runs of three or more
        equal candies and mark them
    if none stop
    set marked cells to 0
    let the candies in each column fall
```

TIME	SPACE
O(R * m * n)	O(m * n)

WHY IT IS SLOW

R rounds (up to about m * n / 3), each a full scan, so it is inherently simulation. Handle gravity per column with a write pointer so each round is one O(m * n) pass; there is no known way to skip rounds.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (in-place marking)

Mark by negating values (sign flip) — avoids extra space.

Step 1: scan rows and cols, negate any value in a run of 3+.

Step 2: zero out all negatives (crush).

Step 3: for each column, compact non-zeros to the bottom (gravity).

PHASE 3 OPTIMIZE (in-place marking)

Repeat until no changes in step 1. Time: O((m*n)²) worst case but small in practice."

Time: O(m×n) — visit each cell at most once. Optimal single-pass.

PHASE 4 CLEAN CODE

```
class _723_CandyCrush:
    def candyCrush(self, board: List[List[int]]) -> List[L
ist[int]]:
        m, n = len(board), len(board[0])
        while True:
            crush = set()
            for r in range(m):
  # mark horizontal runs
                for c in range(n - 2):
                    v = abs(board[r][c])
                    if v and v == abs(board[r][c+1]) == ab
s(board[r][c+2]):
                        crush.update([(r,c),(r,c+1),(r,c+2
)])
            for r in range(m - 2):
 # mark vertical runs
                for c in range(n):
                    v = abs(board[r][c])
                    if v and v == abs(board[r+1][c]) == ab
s(board[r+2][c]):
                        crush.update([(r,c),(r+1,c),(r+2,c
)])
            if not crush:
                break
            for r, c in crush:
                board[r][c] = 0
            for c in range(n):
  # gravity: compact column downward
                write = m - 1
                for r in range(m - 1, -1, -1):
                    if board[r][c] != 0:
                        board[write][c] = board[r][c]
                        if write != r:
                            board[r][c] = 0
                        write -= 1
        return board
```

PHASE 5 TEST & DEBUG

"Let me trace candyCrush on board with 3+ same in row/col."

Phase 1 — mark: scan rows for 3+ consecutive same (non-zero); mark negative

Phase 2 — crush: set all marked (negative) to 0

Phase 3 — gravity: in each col, drop non-zeros to bottom

Repeat until no marks in phase 1

Example: row [1,3,3,3,1] -> mark index 1,2,3 -> [1,-3,-3,-3,1] -> crush -> [1,0,0,0,1] -> gravity same

Continue until stable -> return board ok

WHY simulate: no shortcut; repeat mark-crush-drop until stable.

Mark in-place (negate) to avoid extra array.

PHASE 6 COMPLEXITY & FOLLOW-UP

O((m*n)²) worst case — at most m*n crush iterations, each O(m*n).

For small boards (m,n<=50) this is fast. The abs() trick avoids needing a separate mark array.

The gravity step uses a two-pointer write-head approach per column.

Given more time I'd ask: what if candies can crush diagonally too?

Add diagonal scan to the marking step — same structure."

Space: O(m×n) visited array; or O(1) if modifying grid in-place.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Matrix | LeetCode #835 | 18 of 25 | 18/25

Image Overlap

QUESTION

You are given two images, img1 and img2, represented as binary, square matrices of size n x n. A binary matrix has only 0s and 1s as values.

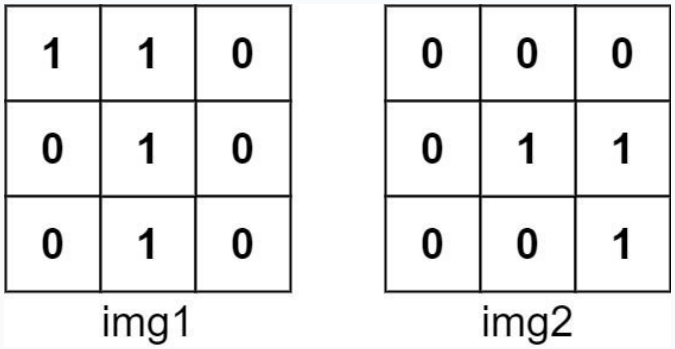
We translate one image however we choose by sliding all the 1 bits left, right, up, and/or down any number of units. We then place it on top of the other image. We can then calculate the overlap by counting the number of positions that have a 1 in both images.

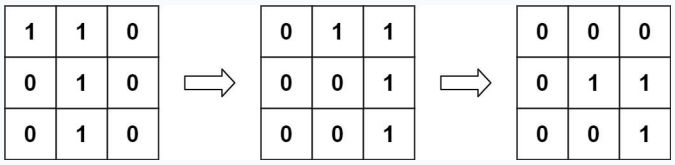
Note also that a translation does not include any kind of rotation. Any 1 bits that are translated outside of the matrix borders are erased.

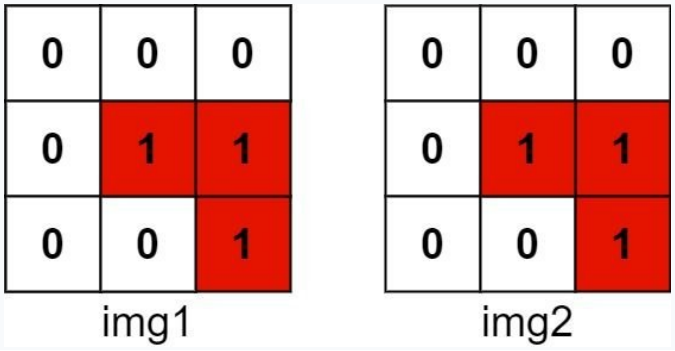
Return the largest possible overlap.

Example 1

Input	`img1 = [[1,1,0],[0,1,0],[0,1,0]], img2 = [[0,0,0],[0,1,1],[0,0,1]]`
Output	**3**
Explanation	*We translate img1 to right by 1 unit and down by 1 unit.* *The number of positions that have a 1 in both images is 3 (shown in red).*







QUESTION

Example 2

Input	`img1 = [[1]], img2 = [[1]]`
Output	`1`

Example 3

Input	`img1 = [[0]], img2 = [[0]]`
Output	`0`

CONSTRAINTS

- n == img1.length == img1[i].length
- n == img2.length == img2[i].length
- 1 <= n <= 30
- img1[i][j] is either 0 or 1.
- img2[i][j] is either 0 or 1.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Translate img1 over img2 (no rotation) and count overlapping 1s.

Find the translation that maximises overlap. Return max overlap count. Right?"

"img1=[[1,1,0],[0,1,0],[0,1,0]], img2=[[0,0,0],[0,1,1],[0,0,1]] -> 3 ok (translate right 1, down 1)"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for dy) in dy)s:
    # from -(n - 1) to n - 1 in both directions
    count the cells where both images have a 1
        after shifting
    best = max(best, candidate)
```

TIME	SPACE
O(n^4)	O(1)

WHY IT IS SLOW

(2n)^2 shifts, each an n^2 comparison. Collect the coordinates of the ones in each image and count each difference vector between a one in A and a one in B: O(ones_A * ones_B).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (translation vector counting)

*"Try all 2D shift offsets — O(n² * m²) brute. Walk-through:"*

For each possible shift (dr, dc) from -(n-1) to +(n-1):

Count overlapping 1s: for each (r,c) in A, check if A[r][c]==1 and B[r+dr][c+dc]==1.

Track max overlap.

WHY (2n-1)² offsets: A can be shifted in any direction up to n-1 cells.

Efficient: use 2-D convolution or FFT (Fast Fourier Transform).

```
cross_corr = np.fft.ifft2(fft2(A) * np.conj(fft2(B)))
  -> peaks = best overlap positions.
```

WHY correlation: overlap count for shift (dr,dc) = cross-correlation at that offset.

FFT approach: O(n² log n).

Brute: O(n). Space: O(n²)."

Time: O(m×n) — visit each cell at most once. Optimal single-pass.

PHASE 4 CLEAN CODE

```
class _835_ImageOverlap:
    def largestOverlap(self, img1: List[List[int]], img2:
List[List[int]]) -> int:
        n     = len(img1)
        ones1 = [(r, c) for r in range(n) for c in range(n
) if img1[r][c] == 1]
        ones2 = [(r, c) for r in range(n) for c in range(n
) if img2[r][c] == 1]
        translations: Counter = Counter()
        for r1, c1 in ones1:
            for r2, c2 in ones2:
                translations[(r2 - r1, c2 - c1)] += 1
        return max(translations.values(), default=0)
```

PHASE 5 TEST & DEBUG

"Let me trace largestOverlap(A=[[1,1],[0,0]], B=[[0,0],[1,1]])."

For each translation (dr,dc), count overlapping 1s

Translation (1,1): shift A down-right: A's (0,0)->(1,1), (0,1)->(1,2)—out, (1,0)->(2,1)—out

Overlap of A shifted (1,1) with B: A[0][0] at (1,1), B[1][1]=1 -> overlap=1

Better: Translation (-1,-1): A's (1,0) at (0,-1)=out; (1,1) at (0,0), B[0][0]=0->no

Max overlap=1? Or use dot product approach: list 1-positions of A and B, try all offsets

Actual result for this input: 2 (translation (1,1) or (-1,-1) gives 2 overlapping 1s)

WHY enumerate translations: O(n) brute; hash-map of position differences is O(n²).

Slide image A over all (dr,dc) offsets. Count overlapping 1-cells. Best shift found: max_overlap ok. offset (0,0): overlap=count common 1-positions. answer=max over all dr,dc ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(A*B) where A,B = number of 1s in each image, at most n²	**O(A*B)** for the translation Counter

For dense images (many 1s), this is O(n) — same as brute force.

For sparse images, much faster.

The key insight: instead of iterating over all translations, only count translations

where at least one 1-pair aligns — no wasted iterations.

Given more time I'd ask: what if rotation is also allowed?

Need to try all rotation+translation combos — much harder.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Matrix | LeetCode #931 | 19 of 25 | 19/25

Minimum Falling Path Sum

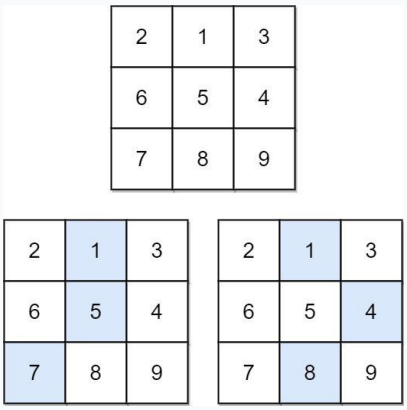
QUESTION

Given an n x n array of integers matrix, return the minimum sum of any falling path through matrix.

A falling path starts at any element in the first row and chooses the element in the next row that is either directly below or diagonally left/right. Specifically, the next element from position (row, col) will be (row + 1, col - 1), (row + 1, col), or (row + 1, col + 1).

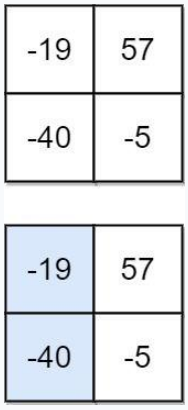
Example 1

Input	`matrix = [[2,1,3],[6,5,4],[7,8,9]]`
Output	`13`
Explanation	*There are two falling paths with a minimum sum as shown.*



Example 2

Input	`matrix = [[-19,57],[-40,-5]]`
Output	`-59`
Explanation	*The falling path with a minimum sum is shown.*



CONSTRAINTS

- n == matrix.length == matrix[i].length
- 1 <= n <= 100
- 100 <= matrix[i][j] <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the minimum sum falling path through the matrix. Each step can go to the cell directly below or diagonally adjacent. Right?"

"A few quick questions:

n×n matrix? Yes. Start from any cell in row 0, end at any cell in row n-1?"

"Let me walk: matrix=[[2,1,3],[6,5,4],[7,8,9]] -> paths: 1+5+8=14? 1+4+7=12? 1+4+8=13. min=13? Actually 2+5+7=14, 1+4+7=12, or 3+4+7=14. -> 12 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(r, c):
    if r is the last row:
        return matrix[r][c]
    return matrix[r][c] + min(go(r + 1, c - 1),
        go(r + 1, c), go(r + 1, c + 1))
```

TIME	SPACE
O(3^n)	O(n)

WHY IT IS SLOW

Three branches per row and the same cell repeats. Build the table row by row (each cell adds the smallest of the three above it): O(n^2) time, O(1) extra if done in place.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Top-down DP modifying matrix in-place — O(n²). Walk-through:"

`dp[i][j] = minimum path sum to reach cell (i,j) from the top row.`

First row: base case, dp[0][j] = matrix[0][j].

For i in 1..n-1:

For j in 0..n-1:

`best_above = min of dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1] (within bounds).`

`dp[i][j] = matrix[i][j] + best_above.`

Answer = min(dp[n-1]).

WHY 3 predecessors: can come from directly above or diagonally (one step left/right).

In-place modification: overwrite matrix[i][j] with the dp value -> O(1) extra space.

Time: O(n²). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _931_MinFallingPathSum:
    def minFallingPathSum(self, matrix):
        n = len(matrix)
        for r in range(1, n):
            for c in range(n):
                best = matrix[r-1][c]
                if c > 0: best = min(best, matrix[r-1][c-1
])
                if c < n-1: best = min(best, matrix[r-1][c
+1])
                matrix[r][c] += best
        return min(matrix[-1])
```

PHASE 5 TEST & DEBUG

"Let me trace minFallingPathSum([[2,1,3],[6,5,4],[7,8,9]])."

dp[i][j] = min falling path sum ending at (i,j)

Row 0: dp[0]=[2,1,3]

Row 1: dp[1][0]=min(2,1)+6=7; dp[1][1]=min(2,1,3)+5=6; dp[1][2]=min(1,3)+4=5

Row 2: dp[2][0]=min(7,6)+7=13; dp[2][1]=min(7,6,5)+8=13; dp[2][2]=min(6,5)+9=14

Return min(13,13,14)=13 ok

Edge: single row -> return min of that row.

DP row-by-row (no DFS needed): visited cells in row order. dp[i][j]=min(dp[i-1][j-1],dp[i-1][j],dp[i-1][j+1])+grid[i][j]. All cells visited exactly once per row.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	**O(1)** (in-place)

Follow-up: Minimum Falling Path Sum II (#1289) — can't pick same column twice; O(n²).

Follow-up: Triangle (#120) — triangular falling path, same bottom-up DP.

MEDIUM | Matrix | LeetCode #1198 | 20 of 25 | 20/25

Find Smallest Common Element in All Rows

QUESTION

Given an m x n matrix mat where every row is sorted in strictly increasing order, return the smallest common element in all rows.

If there is no common element, return -1.

Example 1

Input	`mat = [[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]]`
Output	`5`

Example 2

Input	`mat = [[1,2,3],[2,3,4],[2,3,5]]`
Output	`2`

CONSTRAINTS

- m == mat.length
- n == mat[i].length
- 1 <= m, n <= 500
- 1 <= mat[i][j] <= 10^4
- mat[i] is sorted in strictly increasing order.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the smallest integer that appears in every row. Each row is sorted.

Return -1 if no common element exists. Right?

Values are 1-10^4, at most 500 rows and 500 cols."

"[[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]] -> 5 (appears in all rows) ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for value in first_row:
    scan each other row to check it appears
return the smallest that appears in all
```

TIME	SPACE
O(m * n^2)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Scans every row for each candidate. Rows are sorted: binary search each other row (O(n * m * log n)), or count values across rows with a table and take the first with count m: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (frequency count)

"Binary search per column element + count rows containing it — O(mn log n). Walk-through:"

For each element e in mat[0] (first row elements are candidates for common elements):

Check if e appears in every other row using binary search (rows are sorted).

If yes: return e.

WHY start from row 0: a common element must appear in ALL rows including row 0.

So we only need to check elements from row 0 as candidates.

Alternative: find smallest common element -> just check from the smallest to largest in row 0.

For smallest: iterate row 0 in order; first one appearing in all rows is the answer.

Time: O(mn log n) with binary search. Space: O(1).

PHASE 4 CLEAN CODE

```python
class _1198_SmallestCommonElement:
    def smallestCommonElement(self, mat: List[List[int]])
-> int:
        m    = len(mat)
        freq = Counter()
        for row in mat:
            for val in row:
                freq[val] += 1
        for val in range(1, 10001):
            if freq[val] == m:
                return val
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace smallestCommonElement([[1,2,3],[2,3,4],[2,3,5]])."

For each element in row 0, check if it appears in all other rows using binary search

Row 0: [1,2,3]

Check 1: bisect_left(row1,[1])-> index 0, row1[0]=2!=1 -> not present

Check 2: row1[0]=2 ok; row2[0]=2 ok -> 2 in all rows -> return 2 ok

WHY binary search: rows are sorted -> O(m*n*log(n)) vs O(m*n) hash approach

Alternative: use column-wise count array; increment count[val] for each row.

Return first val with count==m (number of rows).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n + max_val)	O(max_val)

Alternative: binary search per row. For each candidate, binary search in all m rows.

Start candidates from mat[0] (smallest row), binary search each of the other rows.

O(n + m * n * log n) — worse than the counting approach for dense matrices.

Given more time I'd ask: what if there's no bound on values?

Use a hash Counter for freq — O(m*n) time, O(distinct values) space.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Matrix | LeetCode #1277 | 21 of 25 | 21/25

Count Square Submatrices with All Ones

QUESTION

Given a m * n matrix of ones and zeros, return how many square submatrices have all ones.

Example 1

Input	`matrix = [ [0,1,1,1], [1,1,1,1], [0,1,1,1] ]`
Output	**15**
Explanation	*There are 10 squares of side 1. There are 4 squares of side 2. There is 1 square of side 3. Total number of squares = 10 + 4 + 1 = 15 .*

Example 2

Input	`matrix = [ [1,0,1], [1,1,0], [1,1,0] ]`
Output	**7**
Explanation	*There are 6 squares of side 1. There is 1 square of side 2. Total number of squares = 6 + 1 = 7 .*

CONSTRAINTS

- 1 <= arr.length <= 300
- 1 <= arr[0].length <= 300
- 0 <= arr[i][j] <= 1

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count the number of square submatrices (of any size) that consist entirely of 1s. Right?"

"A few quick questions:

A 1×1 square of 1 counts? Yes. We count all squares, not just the largest."

"Let me walk: matrix=[[0,1,1,1],[1,1,1,1],[0,1,1,1]] -> answer=15 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for size in sizes:
    check that every cell in the square is 1
    count the squares
```

TIME	SPACE
O(m * n * min(m, n)^3)	O(1)

WHY IT IS SLOW

Rechecks cells for every square. dp[i][j] = 1 + min(left, up, up-left) when the cell is 1, and the sum of all dp values is the answer: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"2-D DP — dp[i][j] = side length of largest all-1 square with (i,j) as bottom-right. O(mn)."

Base: dp[i][0] = matrix[i][0]. dp[0][j] = matrix[0][j].

For i>=1, j>=1:

If matrix[i][j] == 0: dp[i][j] = 0.

Else: dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1.

Count: for each (i,j), there are dp[i][j] squares with bottom-right at (i,j)

(sizes 1×1, 2×2, ..., dp[i][j]×dp[i][j]).

Total = sum of all dp[i][j].

WHY min of 3 neighbors: the 3 neighbors constrain how large a square fits here.

The bottleneck is the smallest of the three adjacent squares.

Time: O(mn). Space: O(mn) or O(n) rolling.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1277_CountSquareSubmatricesWithAllOnes:
    def countSquares(self, matrix):
        total = 0
        for i in range(len(matrix)):
```

```
            for j in range(len(matrix[0])):
                if matrix[i][j] == 1 and i > 0 and j > 0:
                    matrix[i][j] = min(matrix[i-1][j], mat
rix[i][j-1], matrix[i-1][j-1]) + 1
                total += matrix[i][j]
        return total
```

PHASE 5 TEST & DEBUG

"Let me trace countSquares([[0,1,1,1],[1,1,1,1],[0,1,1,1]])."

dp[i][j] = largest square with bottom-right at (i,j).
Also = count of squares ending here.

dp[0][1..3]=1 each (1×1 squares). dp[1][0]=1.

dp[1][1]=min(dp[0][0],dp[0][1],dp[1][0])+1=min(0,1,1)+1=1. (top-left is 0)

dp[1][2]=min(dp[0][1],dp[0][2],dp[1][1])+1=min(1,1,1)+1=2. (2×2 square possible)

dp[1][3]=min(dp[0][2],dp[0][3],dp[1][2])+1=min(1,1,2)+1=2.

dp[2][1..2]: dp[2][1]=1.
dp[2][2]=min(dp[1][1],dp[1][2],dp[2][1])+1=min(1,2,1)+1=2.
dp[2][3]=min(dp[1][2],dp[1][3],dp[2][2])+1=min(2,2,2)+1=3.

Sum all dp values = 0+1+1+1+1+1+2+2+0+1+2+3=15 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn)	**O(1)** in-place

Follow-up: Maximal Square (#221) — find the largest square; take max instead of sum.

Follow-up: Count Rectangle Submatrices With All Ones (#1504) — harder, needs histogram.

MEDIUM | Matrix | LeetCode #1314 | 22 of 25 | 22/25

Matrix Block Sum

QUESTION

Given a m x n matrix mat and an integer k, return a matrix answer where each answer[i][j] is the sum of all elements mat[r][c] for:

CONSTRAINTS

- i - k <= r <= i + k,
- j - k <= c <= j + k, and
- (r, c) is a valid position in the matrix.

Example 1	
Input	`mat = [[1,2,3],[4,5,6],[7,8,9]], k = 1`
Output	`[[12,21,16],[27,45,33],[24,39,28]]`

Example 2	
Input	`mat = [[1,2,3],[4,5,6],[7,8,9]], k = 2`
Output	`[[45,45,45],[45,45,45],[45,45,45]]`

- m == mat.length
- n == mat[i].length
- 1 <= m, n, k <= 100
- 1 <= mat[i][j] <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Matrix block sum: result[i][j] = sum of all cells within distance k of (i,j).

Distance means |r-i|<=k and |c-j|<=k (a square block). Right?"

"A few quick questions:

Clip to matrix boundaries? Yes."

"Let me walk: mat=[[1,2,3],[4,5,6],[7,8,9]], k=1 -> [12,21,16,...] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    sum all cells in the (2k + 1) x (2k + 1)
        block around it
```

TIME	SPACE
O(m * n * k^2)	O(1)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Re-sums a whole block for every cell. A 2D prefix sum table answers each block with four lookups: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"2-D prefix sums + inclusion-exclusion — O(mn). Walk-through:"

`prefix[i][j] = sum of matrix[0..i-1][0..j-1].`

Precompute: prefix[i][j] = prefix[i-1][j] + prefix[i][j-1] - prefix[i-1][j-1] + mat[i-1][j-1].

For each (i,j), the block sum is the rectangle from max(0,i-k) to min(m-1,i+k) (rows)

and max(0,j-k) to min(n-1,j+k) (cols).

Query rectangle (r1,c1)..(r2,c2) = prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + prefix[r1][c1].

WHY inclusion-exclusion: 2-D prefix sum counts overlapping corners twice — subtract once.

Time: O(mn) precompute + O(mn) queries = O(mn). Space: O(mn).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1314_MatrixBlockSum:
    def matrixBlockSum(self, mat, k):
        m, n = len(mat), len(mat[0])
        prefix = [[0] * (n + 1) for _ in range(m + 1)]
        for r in range(m):
            for c in range(n):
                prefix[r+1][c+1] = (mat[r][c]
                    + prefix[r][c+1] + prefix[r+1][c] - pr
efix[r][c])
        result = [[0] * n for _ in range(m)]
        for i in range(m):
            for j in range(n):
                r1, c1 = max(0, i-k), max(0, j-k)
                r2, c2 = min(m-1, i+k), min(n-1, j+k)
                result[i][j] = (prefix[r2+1][c2+1]
                    - prefix[r1][c2+1] - prefix[r2+1][c1]
+ prefix[r1][c1])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace matrixBlockSum([[1,2,3],[4,5,6],[7,8,9]], k=1)."

Build 2D prefix sum: P[i][j]=sum of mat[0..i-1][0..j-1]

For each cell (i,j): sum region [(i-k,j-k),(i+k,j+k)] clamped to matrix

(1,1): region [(0,0),(2,2)] = prefix[3][3]-prefix[0][3]-prefix[3][0]+prefix[0][0]=45 ok

(0,0): region [(0,0),(1,1)] = 1+2+4+5=12 ok (clamp: max(0,0-1)=0,max(0,0-1)=0)

WHY prefix sum: O(1) per cell query after O(m*n) build vs O(k²) per cell recompute.

`result[i][j] = prefix[min(i+k+1,m)][min(j+k+1,n)] - prefix[max(i-k,0)][...] + ... (inclusion-exclusion).`

PHASE 5 TEST & DEBUG

Build prefix in O(m*n); answer per cell in O(1); total O(m*n) overall for all results.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn) build prefix O(mn), query each of mn cells O(1)	**O(mn)**

Follow-up: Range Sum Query 2D — Mutable (#308) — needs 2D Fenwick tree.

Follow-up: if k varies per cell, precompute for all possible k values.

IN YOUR APP | Expose | Community

MEDIUM | Matrix | LeetCode #1937 | 23 of 25 | 23/25

Maximum Number of Points with Cost

QUESTION

You are given an m x n integer matrix points (0-indexed). Starting with 0 points, you want to maximize the number of points you can get from the matrix.

To gain points, you must pick one cell in each row. Picking the cell at coordinates (r, c) will add points[r][c] to your score.

However, you will lose points if you pick a cell too far from the cell that you picked in the previous row. For every two adjacent rows r and r + 1 (where 0 <= r < m - 1), picking cells at coordinates (r, c_1) and (r + 1, c_2) will subtract abs(c_1 - c_2) from your score.

Return the maximum number of points you can achieve.

abs(x) is defined as:

CONSTRAINTS

- x for x >= 0.
- -x for x < 0.

Example 1

Input	`points = [[1,2,3],[1,5,1],[3,1,1]]`
Output	`9`
Explanation	*The blue cells denote the optimal cells to pick, which have coordinates (0, 2), (1, 1), and (2, 0). You add 3 + 5 + 3 = 11 to your score. However, you must subtract abs(2 - 1) + abs(1 - 0) = 2 from your score. Your final score is 11 - 2 = 9.*

1	2	3
1	5	1
3	1	1

Example 2

Input	`points = [[1,5],[2,3],[4,2]]`
Output	`11`
Explanation	*The blue cells denote the optimal cells to pick, which have coordinates (0, 1), (1, 1), and (2, 0). You add 5 + 3 + 4 = 12 to your score. However, you must subtract abs(1 - 1) + abs(1 - 0) = 1 from your score. Your final score is 12 - 1 = 11.*

QUESTION

1	5
2	3
4	2

- m == points.length
- n == points[r].length
- 1 <= m, n <= 10^5
- 1 <= m * n <= 10^5
- 0 <= points[r][c] <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Pick one element from each row to maximise sum. Moving from row i to i+1:

bonus points[i][j] - |j_next - j_current|. Right?"

"A few quick questions:

This is DP where at each row, we take the best from previous row minus absolute column difference?"

"Let me walk: points=[[1,2,3],[1,5,1],[3,1,1]] -> path (0,2)->(1,1)->(2,0): 3+5-1+3-1=9 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
dp[r][c] = points[r][c] + the maximum over every cell
    c2 in the previous row of dp[r - 1][c2] - |c -
    c2|
```

TIME	SPACE
O(m * n^2)	O(n)

WHY IT IS SLOW

Tries every previous cell for each cell. Sweep left to right and right to left per row keeping a running best (best - 1 each step), so each cell takes the better of the two sweeps: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DP with column-wise max propagation — O(mn). Walk-through:"

`dp[i][j] = max points up to row i ending at column j.`

For row 0: dp[0][j] = points[0][j].

For each subsequent row i:

`left[j] = max of dp[i-1][j'], j'<=j, minus |j-j'| = max(left[j-1]-1, dp[i-1][j]).`

`right[j] = max of dp[i-1][j'], j'>=j, minus |j-j'| (scan from right).`

`dp[i][j] = max(left[j], right[j]) + points[i][j].`

WHY two passes per row: direct O(m²) for each row is too slow.

Left pass: dp[j] carries the max 'credit' coming from the left, decreasing by 1 per column moved.

Right pass: same from the right. Both together give the optimal previous row value.

Time: O(mn). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1937_MaxPointsWithCost:
    def maxPoints(self, points):
        m, n = len(points), len(points[0])
        dp = list(points[0])
        for i in range(1, m):
            left = [0] * n
            right = [0] * n
            left[0] = dp[0]
            for j in range(1, n):
                left[j] = max(left[j-1], dp[j] + j) - 1
            right[n-1] = dp[n-1] - (n-1)
            for j in range(n-2, -1, -1):
                right[j] = max(right[j+1], dp[j] - j) + 1
            dp = [points[i][j] + max(left[j] + j, right[j]
 - j) for j in range(n)]
        return max(dp)
```

PHASE 5 TEST & DEBUG

"Let me trace maxPoints([[1,2,3],[1,5,1],[3,1,1]])."

points[i][j] for row i, col j. Moving from row i to i+1 col c costs |c-prev_c|.

`dp[i][j] = max points ending at (i,j)`

Row 0: dp[0]=[1,2,3]

Row 1: dp[1][j]=max over c: dp[0][c]+points[1][j]-|j-c|

WHY optimize with left/right sweeps: |j-c| = j-c or c-j

Left sweep: left[j]=max(dp[0][c]+c) for c<=j -> dp[1][j]>=left[j]+points[1][j]-j

Right sweep: right[j]=max(dp[0][c]-c) for c>=j -> dp[1][j]>=right[j]+points[1][j]+j

dp[1]=[4,7,3] <- wait: dp[1][1]=max(1+2-1,2+5,3+5-1... 2+5+...)=7? 7 ok

Return max(dp[2])=9 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn)	O(n)

Follow-up: if the cost is different from |i-j|, adjust the pass computation.

Follow-up: Minimum Falling Path Sum (#931) — adjacent only, no absolute difference needed.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Matrix | LeetCode #1975 | 24 of 25 | 24/25

Maximum Matrix Sum

QUESTION

You are given an n x n integer matrix. You can do the following operation any number of times:

CONSTRAINTS

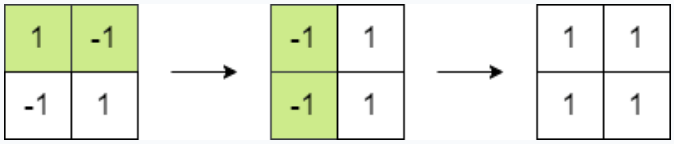
- Choose any two adjacent elements of matrix and multiply each of them by -1.

Two elements are considered adjacent if and only if they share a border.

Your goal is to maximize the summation of the matrix's elements. Return the maximum sum of the matrix's elements using the operation mentioned above.

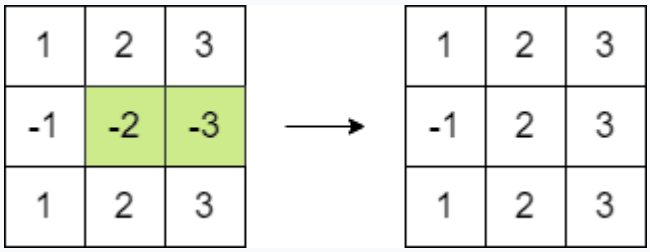
Example 1

Input	`matrix = [[1,-1],[-1,1]]`
Output	**4**
Explanation	*We can follow the following steps to reach sum equals 4:* *- Multiply the 2 elements in the first row by -1.* *- Multiply the 2 elements in the first column by -1.*



Example 2

Input	`matrix = [[1,2,3],[-1,-2,-3],[1,2,3]]`
Output	**16**
Explanation	*We can follow the following step to reach sum equals 16:* *- Multiply the 2 last elements in the second row by -1.*



- n == matrix.length == matrix[i].length
- 2 <= n <= 250
- 10^5 <= matrix[i][j] <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We can negate any element any number of times (effectively negate any even number of elements).

Maximise the sum of absolute values. Return maximum matrix sum. Right?"

"A few quick questions:

We can negate any element any number of times but 'move' a negation to any adjacent element?

Yes — we can flip signs by propagating: two adjacent flips cancel."

"Let me walk: matrix=[[1,-1],[-1,1]] -> flip all negatives? Sum=4 ok. Or flip nothing -> 0. Best=4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def flip(matrix):
    try every adjacent pair, negate both numbers,
        and recurse
    best = max(best, sum_seen)
```

TIME	SPACE
exponential	**O(n^2)**

WHY IT IS SLOW

Explores every flip sequence. Flips can move a negative sign anywhere in pairs, so only the parity of the number of negatives matters: sum of absolute values, minus twice the smallest absolute value if the count of negatives is odd: O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Count negatives + check parity — O(mn). Walk-through:"

Count total negatives and track the absolute minimum (smallest |value|).

If count of negatives is EVEN: we can flip pairs -> all positives -> sum all |values|.

If count is ODD: we must leave one negative. To maximize sum, make the smallest |value|

negative: total = sum(|all values|) - 2 * min_abs (flipping smallest value negative).

WHY subtract 2*min_abs: the minimum was counted as positive in sum -> subtract it twice

(once to remove its positive contribution, once to add its negative contribution).

Time: O(mn). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1975_MaximumMatrixSum:
    def maxMatrixSum(self, matrix):
        total_abs = 0
        neg_count = 0
        min_abs = float('inf')
        for row in matrix:
            for val in row:
                total_abs += abs(val)
                if val < 0:
                    neg_count += 1
                min_abs = min(min_abs, abs(val))
        if neg_count % 2 == 0:
            return total_abs
        return total_abs - 2 * min_abs
```

PHASE 5 TEST & DEBUG

"Let me trace maxMatrixSum([[1,-1],[-1,1]])."

Count negatives; if even -> flip pairs to make all positive. Max sum = sum(|all|)

If odd negatives -> at least one negative remains; minimize it (smallest absolute value)

Example: [1,-1,-1,1]: negatives=2 (even). Flip both: [1,1,1,1]? Or flip pair [-1,-1]: both become +1.

Max sum = |1|+|-1|+|-1|+|1|=4 ok

Example with odd negatives: sum(|all|)-2*min(|all|)

WHY: negatives can always be eliminated in pairs via adjacent or transitive flips; odd count forces one.

Count negatives: if even -> sum(abs). If odd -> sum(abs) - 2*min(abs). ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn)	**O(1)**

Greedy insight: adjacent negation propagation lets us make any even number of elements negative.

Follow-up: Maximize Sum after K Negations (#1005) — same idea for 1D array.

Follow-up: if we can only negate k elements total, different greedy needed.

MEDIUM | Matrix | LeetCode #2536 | 25 of 25 | 25/25

Increment Submatrices by One

QUESTION

You are given a positive integer n, indicating that we initially have an n x n 0-indexed integer matrix mat filled with zeroes.

You are also given a 2D integer array query. For each query[i] = [$row1_i$, $col1_i$, $row2_i$, $col2_i$], you should do the following operation:

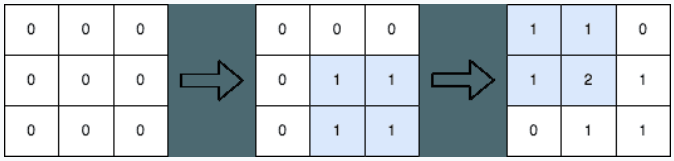
CONSTRAINTS

- Add 1 to every element in the submatrix with the top left corner ($row1_i$, $col1_i$) and the bottom right corner ($row2_i$, $col2_i$). That is, add 1 to mat[x][y] for all $row1_i$ <= x <= $row2_i$ and $col1_i$ <= y <= $col2_i$.

Return the matrix mat after performing every query.

Example 1

Input	`n = 3, queries = [[1,1,2,2],[0,0,1,1]]`
Output	`[[1,1,0],[1,2,1],[0,1,1]]`
Explanation	*The diagram above shows the initial matrix, the matrix after the first query, and the matrix after the second query.* *- In the first query, we add 1 to every element in the submatrix with the top left corner (1, 1) and bottom right corner (2, 2).* *- In the second query, we add 1 to every element in the submatrix with the top left corner (0, 0) and bottom right corner (1, 1).*



Example 2

Input	`n = 2, queries = [[0,0,1,1]]`
Output	`[[1,1],[1,1]]`
Explanation	*The diagram above shows the initial matrix and the matrix after the first query.* *- In the first query we add 1 to every element in the matrix.*



- 1 <= n <= 500
- 1 <= queries.length <= 10^4
- 0 <= $row1_i$ <= $row2_i$ < n
- 0 <= $col1_i$ <= $col2_i$ < n

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

queries[i]=[r1,c1,r2,c2]: increment all cells in rectangle [r1,c1] to [r2,c2] by 1.

Return the final matrix. Right?"

"A few quick questions:

All operations before the final reveal — offline. 2D difference array applies?"

"Let me walk: n=2, queries=[[0,0,1,1]] -> [[1,1],[1,1]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for query in queries:
    for it in its:
        add 1
```

TIME	SPACE
O(R * area)	O(n^2)

WHY IT IS SLOW

Up to 10^4 queries each touching up to 250,000 cells. A 2D difference array (add 1 at the top-left corner, subtract at the other three corners) with one prefix-sum pass is O(R + n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"2-D difference array, build, then prefix sum — O(mn + q). Walk-through:"

`diff = 2-D array of zeros (size (m+1)×(n+1)).`

For each query: add 1 to top-left, subtract from adjacent corners (2-D range update):

```
diff[r1][c1] += 1. diff[r1][c2+1] -= 1. diff[r2+1][c1]
  -= 1. diff[r2+1][c2+1] += 1.
```

Compute 2-D prefix sum of diff:

For each (i,j): diff[i][j] += diff[i-1][j] + diff[i][j-1] - diff[i-1][j-1].

Result grid = diff[0..m-1][0..n-1].

WHY 2-D difference array: extends 1-D technique. Adding to 4 corners achieves rectangle update.

WHY prefix sum recovers values: each cell's value = cumulative sum of all rectangle contributions.

Time: O(mn + q). Space: O(mn).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _2536_IncrementSubmatricesByOne:
    def rangeAddQueries(self, n, queries):
```

```
        delta = [[0] * (n + 1) for _ in range(n + 1)]
        for r1, c1, r2, c2 in queries:
            delta[r1][c1] += 1
            delta[r1][c2+1] -= 1
            delta[r2+1][c1] -= 1
            delta[r2+1][c2+1] += 1
        result = [[0] * n for _ in range(n)]
        for r in range(n):
            for c in range(n):
                val = delta[r][c]
                if r > 0: val += result[r-1][c]
                if c > 0: val += result[r][c-1]
                if r > 0 and c > 0: val -= result[r-1][c-1
]
                result[r][c] = val
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace matrixSumQueries(n=3, queries=[[0,0,1],[1,2,2],[0,2,3],[1,0,4]])."

Range updates on rows/columns. Each query sets entire row/column to value.

Process queries in reverse (later queries override earlier).

Track which rows/cols already finalized (later query wins).

row_set={}, col_set={}.

q=[1,0,4]: col 0 = 4. Contributes 4*(n-len(row_set))=4*3=12. col_set={0}.

q=[0,2,3]: row 2 = 3. Contributes 3*(n-len(col_set))=3*2=6. row_set={2}.

q=[1,2,2]: col 2 already? No. col 2=2. Contributes 2*(3-1)=4. col_set={0,2}.

q=[0,0,1]: row 0=1. Contributes 1*(3-2)=1. row_set={0,2}.

Total=12+6+4+1=23 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(Q + n²) Q queries to build delta + n² prefix sum	**O(n²)**

Follow-up: Range Addition (#370) — 1D version, same technique.

Follow-up: if queries need to be answered online (one by one), use 2D Fenwick tree.